1. Real-Time Inventory Tracking System

Description:
Develop a system to track real-time inventory levels using structures for item details and unions for variable attributes (e.g., weight, volume). Use const pointers for immutable item codes and double pointers for managing dynamic inventory arrays.

Specifications:

- Structure: Item details (ID, name, category).
- Union: Attributes (weight, volume).
- const Pointer: Immutable item codes.
- Double Pointers: Dynamic inventory management.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Item
{
   const char *itemCode;
   char name[50];
   char category[20];
   union
   {
      float weight;
      float volume;
   } attribute;
   char attributeType; // 'W' or 'V'
};
```

```
void addItem(struct Item ***inventory, int *count);
void displayItems(struct Item **inventory, int count);

int main()
{
    struct Item **inventory = NULL;
    int count = 0, option;
    do
    {
        printf("\n--- Real-time Inventory Tracking System ---");
        printf("\n1. Add Item\n2. Display Items\n3. Exit\n");
        printf("Enter the option: ");
        scanf("%d", &option);
        switch(option)
        {
            case 1: addItem(&inventory, &count); break;
            case 2: displayItems(inventory, count); break;
            case 3: printf("Exit the system\n"); break;
            default: printf("Invalid option\n");
        }
    } while(option != 3);
    for (int i = 0; i < count; i++)
    {
        free((void *)inventory[i]->itemCode);
        free(inventory[i]);
    }
    free(inventory);
```

```
    return 0;
}

void addItem(struct Item ***inventory, int *count)
{
    struct Item *newItem = (struct Item *)malloc(sizeof(struct Item));
    if (!newItem)
    {
        printf("Memory allocation failed\n");
        return;
    }
    char code[10];
    printf("Enter Item Code: ");
    scanf(" %s", code);
    char *codeCopy = (char *)malloc((strlen(code) + 1) * sizeof(char));
    if (!codeCopy)
    {
        printf("Memory allocation failed for item code.\n");
        free(newItem);
        return;
    }
    strcpy(codeCopy, code);
    newItem->itemCode = codeCopy;
    printf("Enter Name: ");
    scanf("%s", newItem->name);
    printf("Enter Category: ");
    scanf("%s", newItem->category);
```

```
printf("Attribute Type (W for Weight, V for Volume): ");
scanf(" %c", &newItem->attributeType);
if (newItem->attributeType == 'W')
{
    printf("Enter Weight: ");
    scanf("%f", &newItem->attribute.weight);
}
else if (newItem->attributeType == 'V')
{
    printf("Enter Volume: ");
    scanf("%f", &newItem->attribute.volume);
}
else
{
    printf("Invalid attribute type\n");
    free((void *)newItem->itemCode);
    free(newItem);
    return;
}
struct Item **temp = (struct Item **)realloc(*inventory, (*count + 1)
                                                * sizeof(struct Item *));
if (!temp)
{
    printf("Memory reallocation failed\n");
    free((void *)newItem->itemCode);
    free(newItem);
    return;
```

```
    }
    *inventory = temp;
    (*inventory)[(*count)++] = newItem;
    printf("Item added successfully\n");
}

void displayItems(struct Item **inventory, int count)
{
    if (inventory == NULL || count == 0)
    {
        printf("\nNo items in the inventory to display\n");
        return;
    }
    printf("\n--- Inventory Items ---\n");
    for (int i = 0; i < count; i++)
    {
        printf("\nItem %d:\n", i + 1);
        printf("Item Code: %s\n", inventory[i]->itemCode);
        printf("Name: %s\n", inventory[i]->name);
        printf("Category: %s\n", inventory[i]->category);
        if (inventory[i]->attributeType == 'W')
            printf("Weight: %.2f kg\n", inventory[i]->attribute.weight);
        else
            printf("Volume: %.2f m³\n", inventory[i]->attribute.volume);
    }
}
```

2. Dynamic Route Management for Logistics

Description:
Create a system to dynamically manage shipping routes using structures for route data and unions for different modes of transport. Use const pointers for route IDs and double pointers for managing route arrays.

Specifications:

- Structure: Route details (ID, start, end).
- Union: Transport modes (air, sea, land).
- const Pointer: Read-only route IDs.
- Double Pointers: Dynamic route allocation.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Route
{
   const char *routeID;
   char start[50];
   char end[50];
   union
   {
      float airDistance;
      float seaDistance;
      float landDistance;
   } transportMode;
   char modeType;       // 'A' for Air, 'S' for Sea, 'L' for Land
};
```

```
void addRoute(struct Route ***routes, int *count);
void displayRoutes(struct Route **routes, int count);

int main()
{
    struct Route **routes = NULL;
    int count = 0, option;
    do
    {
        printf("\n--- Dynamic Route Management System ---");
        printf("\n1. Add Route\n2. Display Routes\n3. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &option);
        switch(option)
        {
            case 1: addRoute(&routes, &count); break;
            case 2: displayRoutes(routes, count); break;
            case 3: printf("Exiting the system\n"); break;
            default: printf("Invalid option\n");
        }
    } while (option != 3);
    for (int i = 0; i < count; i++)
    {
        free((void *)routes[i]->routeID);
        free(routes[i]);
    }
    free(routes);
```

```
    return 0;
}

void addRoute(struct Route ***routes, int *count)
{
    struct Route *newRoute = (struct Route *)malloc(sizeof(struct Route));
    if (!newRoute)
    {
        printf("Memory allocation failed for new route.\n");
        return;
    }
    char id[20];
    printf("Enter Route ID: ");
    scanf("%s", id);
    char *idCopy = (char *)malloc((strlen(id) + 1) * sizeof(char));
    if (!idCopy)
    {
        printf("Memory allocation failed for route ID\n");
        free(newRoute);
        return;
    }
    strcpy(idCopy, id);
    newRoute->routeID = idCopy;
    printf("Enter Starting Location: ");
    scanf("%s", newRoute->start);
    printf("Enter Ending Location: ");
    scanf("%s", newRoute->end);
```

```
printf("Enter Transport Mode (A for Air, S for Sea, L for Land): ");
scanf(" %c", &newRoute->modeType);
if (newRoute->modeType == 'A')
{
   printf("Enter Air Distance: ");
   scanf("%f", &newRoute->transportMode.airDistance);
}
else if (newRoute->modeType == 'S')
{
   printf("Enter Sea Distance: ");
   scanf("%f", &newRoute->transportMode.seaDistance);
}
else if (newRoute->modeType == 'L')
{
   printf("Enter Land Distance: ");
   scanf("%f", &newRoute->transportMode.landDistance);
}
else
{
   printf("Invalid transport mode\n");
   free((void *)newRoute->routeID);
   free(newRoute);
   return;
}
struct Route **temp = (struct Route **)realloc(*routes, (*count + 1)
                                                * sizeof(struct Route *));
if (!temp)
```

```
    {
        printf("Memory reallocation failed\n");
        free((void *)newRoute->routeID);
        free(newRoute);
        return;
    }
    *routes = temp;
    (*routes)[(*count)++] = newRoute;
    printf("Route added successfully\n");
}

void displayRoutes(struct Route **routes, int count)
{
    if (routes == NULL || count == 0)
    {
        printf("\nNo routes to display\n");
        return;
    }
    printf("\n--- Route Details ---\n");
    for (int i = 0; i < count; i++)
    {
        printf("\nRoute %d:\n", i + 1);
        printf("Route ID: %s\n", routes[i]->routeID);
        printf("Start Location: %s\n", routes[i]->start);
        printf("End Location: %s\n", routes[i]->end);
        if (routes[i]->modeType == 'A')
            printf("Transport Mode: Air\nDistance: %.2f km\n",
```

```
                                    routes[i]->transportMode.airDistance);
        else if (routes[i]->modeType == 'S')
            printf("Transport Mode: Sea\nDistance: %.2f nautical miles\n",
                                    routes[i]->transportMode.seaDistance);
        else if (routes[i]->modeType == 'L')
            printf("Transport Mode: Land\nDistance: %.2f km\n",
                                    routes[i]->transportMode.landDistance);
        else
            printf("Unknown transport mode\n");
    }
}
```

3. Fleet Maintenance and Monitoring

Description:
Develop a fleet management system using structures for vehicle details and unions for status (active, maintenance). Use const pointers for vehicle identifiers and double pointers to manage vehicle records.

Specifications:

- Structure: Vehicle details (ID, type, status).
- Union: Status (active, maintenance).
- const Pointer: Vehicle IDs.
- Double Pointers: Dynamic vehicle list management.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```
struct Vehicle
{
   const char *vehicleID;
   char type[50];
   union
   {
      int activeStatus;
      int maintenanceStatus;
   } status;
   char statusType; // Truck, Car, Bus
};

void addVehicle(struct Vehicle ***vehicles, int *count);
void displayVehicles(struct Vehicle **vehicles, int count);

int main()
{
   struct Vehicle **vehicles = NULL;
   int count = 0, option;
   do
   {
      printf("\n--- Fleet Management System ---");
      printf("\n1. Add Vehicle\n2. Display Vehicles\n3. Exit\n");
      printf("Enter the option: ");
      scanf("%d", &option);
      switch(option)
      {
```

```
            case 1: addVehicle(&vehicles, &count); break;
            case 2: displayVehicles(vehicles, count); break;
            case 3: printf("Exiting the system\n"); break;
            default: printf("Invalid option\n");
        }
    } while (option != 3);
    for (int i = 0; i < count; i++)
    {
        free((void *)vehicles[i]->vehicleID);
        free(vehicles[i]);
    }
    free(vehicles);
    return 0;
}

void addVehicle(struct Vehicle ***vehicles, int *count)
{
    struct Vehicle *newVehicle = (struct Vehicle *)malloc(sizeof(struct Vehicle));
    if (!newVehicle)
    {
        printf("Memory allocation failed\n");
        return;
    }
    char id[20];
    printf("Enter Vehicle ID: ");
    scanf("%s", id);
    char *idCopy = (char *)malloc((strlen(id) + 1) * sizeof(char));
```

```
if (!idCopy)
{
    printf("Memory allocation failed for vehicle ID\n");
    free(newVehicle);
    return;
}
strcpy(idCopy, id);
newVehicle->vehicleID = idCopy;
printf("Enter Vehicle Type: ");
scanf("%s", newVehicle->type);
printf("Enter Vehicle Status (A for Active, M for Maintenance): ");
scanf(" %c", &newVehicle->statusType);
if (newVehicle->statusType == 'A')
    newVehicle->status.activeStatus = 1;
else if (newVehicle->statusType == 'M')
    newVehicle->status.maintenanceStatus = 1;
else
{
    printf("Invalid status type\n");
    free((void *)newVehicle->vehicleID);
    free(newVehicle);
    return;
}
struct Vehicle **temp = (struct Vehicle **)realloc(*vehicles, (*count + 1)
                                                    * sizeof(struct Vehicle *));
if (!temp)
{
```

```
        printf("Memory reallocation failed\n");
        free((void *)newVehicle->vehicleID);
        free(newVehicle);
        return;
    }
    *vehicles = temp;
    (*vehicles)[(*count)++] = newVehicle;
    printf("Vehicle added successfully\n");
}

void displayVehicles(struct Vehicle **vehicles, int count)
{
    if (vehicles == NULL || count == 0)
    {
        printf("\nNo vehicles to display\n");
        return;
    }
    printf("\n--- Vehicle Details ---\n");
    for (int i = 0; i < count; i++)
    {
        printf("\nVehicle %d:\n", i + 1);
        printf("Vehicle ID: %s\n", vehicles[i]->vehicleID);
        printf("Vehicle Type: %s\n", vehicles[i]->type);
        if (vehicles[i]->statusType == 'A')
            printf("Status: Active\n");
        else if (vehicles[i]->statusType == 'M')
            printf("Status: Maintenance\n");
```

```
        else
            printf("Unknown status\n");
    }
}
```

4. Logistics Order Processing Queue

Description:
Implement an order processing system using structures for order details and unions for payment methods. Use const pointers for order IDs and double pointers for dynamic order queues.

Specifications:

- Structure: Order details (ID, customer, items).
- Union: Payment methods (credit card, cash).
- const Pointer: Order IDs.
- Double Pointers: Dynamic order queue.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Order
{
    const char *orderID;
    char customer[50];
    char items[100];
    union
    {
```

```
        char creditCard[20];
        float cashAmount;
    } paymentMethod;
    char paymentType;      // 'CC' for Credit Card, 'C' for Cash
};

void addOrder(struct Order ***orders, int *count);
void displayOrders(struct Order **orders, int count);

int main()
{
    struct Order **orders = NULL;
    int count = 0, option;
    do
    {
        printf("\n--- Logistics Order Processing System ---");
        printf("\n1. Add Order\n2. Display Orders\n3. Exit\n");
        printf("Enter the choice: ");
        scanf("%d", &option);
        switch(option)
        {
            case 1: addOrder(&orders, &count); break;
            case 2: displayOrders(orders, count); break;
            case 3: printf("Exiting the system\n"); break;
            default: printf("Invalid option\n");
        }
    } while (option != 3);
```

```
    for (int i = 0; i < count; i++)
    {
        free((void *)orders[i]->orderID);
        free(orders[i]);
    }
    free(orders);
    return 0;
}

void addOrder(struct Order ***orders, int *count)
{
    struct Order *newOrder = (struct Order *)malloc(sizeof(struct Order));
    if (!newOrder)
    {
        printf("Memory allocation failed\n");
        return;
    }
    char id[20];
    printf("Enter Order ID: ");
    scanf("%s", id);
    char *idCopy = (char *)malloc((strlen(id) + 1) * sizeof(char));
    if (!idCopy)
    {
        printf("Memory allocation failed for order ID\n");
        free(newOrder);
        return;
    }
```

```
strcpy(idCopy, id);
newOrder->orderID = idCopy;
printf("Enter Customer Name: ");
scanf(" %[^\n]%*c", newOrder->customer);
printf("Enter Items in the Order: ");
scanf(" %[^\n]%*c", newOrder->items);
printf("Enter Payment Method (CC for Credit Card, C for Cash): ");
scanf(" %c", &newOrder->paymentType);
if (newOrder->paymentType == 'CC')
{
   printf("Enter Credit Card Number: ");
   scanf("%s", newOrder->paymentMethod.creditCard);
}
else if (newOrder->paymentType == 'C')
{
   printf("Enter Cash Amount: ");
   scanf("%f", &newOrder->paymentMethod.cashAmount);
}
else
{
   printf("Invalid payment method\n");
   free((void *)newOrder->orderID);
   free(newOrder);
   return;
}
struct Order **temp = (struct Order **)realloc(*orders, (*count + 1)
                                                * sizeof(struct Order *));
```

```
    if (!temp)
    {
        printf("Memory reallocation failed\n");
        free((void *)newOrder->orderID);
        free(newOrder);
        return;
    }
    *orders = temp;
    (*orders)[(*count)++] = newOrder;
    printf("Order added successfully\n");
}

void displayOrders(struct Order **orders, int count)
{
    if (orders == NULL || count == 0)
    {
        printf("\nNo orders to display\n");
        return;
    }
    printf("\n--- Order Details ---\n");
    for (int i = 0; i < count; i++)
    {
        printf("\nOrder %d:\n", i + 1);
        printf("Order ID: %s\n", orders[i]->orderID);
        printf("Customer: %s\n", orders[i]->customer);
        printf("Items: %s\n", orders[i]->items);
        if (orders[i]->paymentType == 'CC')
```

```
            printf("Payment Method: Credit Card\nCard Number: %s\n",
                                        orders[i]->paymentMethod.creditCard);
        else if (orders[i]->paymentType == 'C')
            printf("Payment Method: Cash\nAmount: %.2f\n",
                                    orders[i]->paymentMethod.cashAmount);
        else
            printf("Unknown payment method\n");
    }
}
```

5. Shipment Tracking System

Description:
Develop a shipment tracking system using structures for shipment details and unions for tracking events. Use const pointers to protect tracking numbers and double pointers to handle dynamic shipment lists.

Specifications:

- Structure: Shipment details (tracking number, origin, destination).
- Union: Tracking events (dispatched, delivered).
- const Pointer: Tracking numbers.
- Double Pointers: Dynamic shipment tracking.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Shipment
{
```

```
    const char *trackingNumber;
    char origin[50];
    char destination[50];
    union
    {
        char dispatchedDate[20];
        char deliveryDate[20];
    } trackingEvent;
    char eventType;   // 'DP' for Dispatched, 'DL' for Delivered
};

void addShipment(struct Shipment ***shipments, int *count);
void displayShipments(struct Shipment **shipments, int count);

int main()
{
    struct Shipment **shipments = NULL;
    int count = 0, option;
    do
    {
        printf("\n--- Shipment Tracking System ---");
        printf("\n1. Add Shipment\n2. Display Shipments\n3. Exit\n");
        printf("Enter the option: ");
        scanf("%d", &option);
        switch(option)
        {
            case 1: addShipment(&shipments, &count); break;
```

```
            case 2: displayShipments(shipments, count); break;
            case 3: printf("Exiting the system\n"); break;
            default: printf("Invalid option\n");
        }
    } while (option != 3);
    for (int i = 0; i < count; i++)
    {
        free((void *)shipments[i]->trackingNumber);
        free(shipments[i]);
    }
    free(shipments);
    return 0;
}

void addShipment(struct Shipment ***shipments, int *count)
{
    struct Shipment *newShipment = (struct Shipment *)
                                        malloc(sizeof(struct Shipment));
    if (!newShipment)
    {
        printf("Memory allocation failed\n");
        return;
    }
    char trackingID[20];
    printf("Enter Tracking Number: ");
    scanf("%s", trackingID);
    char *trackingIDCopy = (char *)malloc((strlen(trackingID) + 1)
```

```
                                                    * sizeof(char));
if (!trackingIDCopy)
{
   printf("Memory allocation failed for tracking number\n");
   free(newShipment);
   return;
}
strcpy(trackingIDCopy, trackingID);
newShipment->trackingNumber = trackingIDCopy;
printf("Enter Origin Location: ");
scanf(" %[^\n]", newShipment->origin);
printf("Enter Destination Location: ");
scanf(" %[^\n]", newShipment->destination);
printf("Enter Tracking Event (DP for Dispatched, DL for Delivered): ");
scanf(" %c", &newShipment->eventType);
if (newShipment->eventType == 'DP')
{
   printf("Enter Dispatch Date: ");
   scanf("%s", newShipment->trackingEvent.dispatchedDate);
}
else if (newShipment->eventType == 'DL')
{
   printf("Enter Delivery Date: ");
   scanf("%s", newShipment->trackingEvent.deliveryDate);
}
else
{
```

```
        printf("Invalid event type\n");
        free((void *)newShipment->trackingNumber);
        free(newShipment);
        return;
    }
    struct Shipment **temp = (struct Shipment **)realloc
                                (*shipments, (*count + 1) * sizeof(struct Shipment *));
    if (!temp)
    {
        printf("Memory reallocation failed\n");
        free((void *)newShipment->trackingNumber);
        free(newShipment);
        return;
    }
    *shipments = temp;
    (*shipments)[(*count)++] = newShipment;
    printf("Shipment added successfully\n");
}

void displayShipments(struct Shipment **shipments, int count)
{
    if (shipments == NULL || count == 0)
    {
        printf("\nNo shipments to display\n");
        return;
    }
    printf("\n--- Shipment Details ---\n");
```

```
    for (int i = 0; i < count; i++)
    {
        printf("\nShipment %d:\n", i + 1);
        printf("Tracking Number: %s\n", shipments[i]->trackingNumber);
        printf("Origin: %s\n", shipments[i]->origin);
        printf("Destination: %s\n", shipments[i]->destination);
        if (shipments[i]->eventType == 'DP')
            printf("Event: Dispatched\nDispatch Date: %s\n",
                        shipments[i]->trackingEvent.dispatchedDate);
        else if (shipments[i]->eventType == 'DL')
            printf("Event: Delivered\nDelivery Date: %s\n",
                                shipments[i]->trackingEvent.deliveryDate);
        else
            printf("Unknown event type\n");
    }
}
```

6. Real-Time Traffic Management for Logistics

Description:
Create a system to manage real-time traffic data for logistics using structures for traffic nodes and unions for traffic conditions. Use const pointers for node identifiers and double pointers for dynamic traffic data storage.

Specifications:

- Structure: Traffic node details (ID, location).
- Union: Traffic conditions (clear, congested).
- const Pointer: Node IDs.
- Double Pointers: Dynamic traffic data management.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct TrafficNode
{
    const int *nodeID;
    char location[50];
    union
    {
        char *condition;
    } trafficCondition;
};

void addNode(struct TrafficNode **trafficData, int *nodeCount);
void displayTrafficData(struct TrafficNode *trafficData, int nodeCount);

int main()
{
    struct TrafficNode *trafficData = NULL;
    int nodeCount = 0, option;
    do
    {
        printf("\nReal-Time Traffic Management System\n");
        printf("1. Add Traffic Node\n");
        printf("2. Display Traffic Data\n");
        printf("3. Exit\n");
```

```
        printf("Enter the option: ");
        scanf("%d", &option);
        switch (option)
        {
            case 1: addNode(&trafficData, &nodeCount); break;
            case 2: displayTrafficData(trafficData, nodeCount); break;
            case 3: for (int i = 0; i < nodeCount; i++)
                    {
                        free((int *)trafficData[i].nodeID);
                        free(trafficData[i].trafficCondition.condition);
                    }
                    free(trafficData);
                    printf("Exiting the logistics\n");
                    break;
            default: printf("Invalid option\n");
        }
    } while(option != 3);
    return 0;
}

void addNode(struct TrafficNode **trafficData, int *nodeCount)
{
    int tempNodeID;
    struct TrafficNode newNode;
    printf("Enter Node ID: ");
    scanf("%d", &tempNodeID);
    int *nodeIDPtr = (int *)malloc(sizeof(int));
```

```
    *nodeIDPtr = tempNodeID;
    newNode.nodeID = nodeIDPtr;
    printf("Enter Location: ");
    scanf(" %[^\n]", newNode.location);
    newNode.trafficCondition.condition = (char *)malloc(20 * sizeof(char));
    printf("Enter traffic condition (clear or congested): ");
    scanf(" %[^\n]", newNode.trafficCondition.condition);
    *trafficData = (struct TrafficNode *)realloc(*trafficData, (*nodeCount + 1)
                                * sizeof(struct TrafficNode));
    (*trafficData)[*nodeCount] = newNode;
    (*nodeCount)++;
    printf("Traffic node added successfully\n");
}

void displayTrafficData(struct TrafficNode *trafficData, int nodeCount)
{
    if (nodeCount == 0)
    {
        printf("No traffic data available\n");
        return;
    }
    printf("\nTraffic Data for Logistics\n");
    for (int i = 0; i < nodeCount; i++)
    {
        printf("Node ID: %d\n", *(trafficData[i].nodeID));
        printf("Location: %s\n", trafficData[i].location);
        printf("Traffic Condition: %s\n", trafficData[i].trafficCondition.condition);
```

```
    }
}
```

7. Warehouse Slot Allocation System

Description:
Design a warehouse slot allocation system using structures for slot details and unions for item types. Use const pointers for slot identifiers and double pointers for dynamic slot management.

Specifications:

- Structure: Slot details (ID, location, size).
- Union: Item types (perishable, non-perishable).
- const Pointer: Slot IDs.
- Double Pointers: Dynamic slot allocation.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct WarehouseSlot
{
    const int *slotID;
    char location[50];
    float size;
    union
    {
        char *itemType;
    } itemDetails;
```

```
};

void allocateSlot(struct WarehouseSlot **slots, int *slotCount);
void displaySlotAllocation(struct WarehouseSlot *slots, int slotCount);

int main()
{
   struct WarehouseSlot *slots = NULL;
   int slotCount = 0;
   int option;
   do
   {
      printf("\nWarehouse Slot Allocation System\n");
      printf("1. Allocate Slot\n");
      printf("2. Display Slot Allocation\n");
      printf("3. Exit\n");
      printf("Enter the  option: ");
      scanf("%d", &option);
      switch (option)
      {
         case 1: allocateSlot(&slots, &slotCount); break;
         case 2: displaySlotAllocation(slots, slotCount); break;
         case 3: for (int i = 0; i < slotCount; i++)
                 {
                    free((int *)slots[i].slotID);
                    free(slots[i].itemDetails.itemType);
                 }
```

```
                free(slots);
                printf("Exiting the system\n");
                break;
            default: printf("Invalid option\n");
        }
    }while( option != 3);
    return 0;
}

void allocateSlot(struct WarehouseSlot **slots, int *slotCount)
{
    int tempSlotID;
    struct WarehouseSlot newSlot;
    printf("Enter Slot ID: ");
    scanf("%d", &tempSlotID);
    int *slotIDPtr = (int *)malloc(sizeof(int));
    *slotIDPtr = tempSlotID;
    newSlot.slotID = slotIDPtr;
    printf("Enter Slot Location: ");
    scanf(" %[^\n]", newSlot.location);
    printf("Enter Slot Size: ");
    scanf("%f", &newSlot.size);
    newSlot.itemDetails.itemType = (char *)malloc(20 * sizeof(char));
    printf("Enter Item Type (perishable or non-perishable): ");
    scanf(" %[^\n]", newSlot.itemDetails.itemType);
    *slots = (struct WarehouseSlot *)realloc(*slots, (*slotCount + 1) *
                                                    sizeof(struct WarehouseSlot));
```

```
    (*slots)[*slotCount] = newSlot;
    (*slotCount)++;
    printf("Slot allocated successfully\n");
}

void displaySlotAllocation(struct WarehouseSlot *slots, int slotCount)
{
    if (slotCount == 0)
    {
        printf("No slots allocated\n");
        return;
    }
    printf("\nWarehouse Slot Allocation Details\n");
    for (int i = 0; i < slotCount; i++)
    {
        printf("Slot ID: %d\n", *(slots[i].slotID));
        printf("Location: %s\n", slots[i].location);
        printf("Size: %.2f square meters\n", slots[i].size);
        printf("Item Type: %s\n", slots[i].itemDetails.itemType);
    }
}
```

8. Package Delivery Optimization Tool

Description:
Develop a package delivery optimization tool using structures for package details and unions for delivery methods. Use const pointers for package identifiers and double pointers to manage dynamic delivery routes.

Specifications:

- Structure: Package details (ID, weight, destination).
- Union: Delivery methods (standard, express).
- const Pointer: Package IDs.
- Double Pointers: Dynamic route management.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Package
{
   const int *packageID;
   float weight;
   char destination[50];
   union
   {
      char *deliveryMethod;
   } deliveryDetails;
};

void addPackage(struct Package **packages, int *packageCount);
void displayPackageDetails(struct Package *packages, int packageCount);

int main()
{
   struct Package *packages = NULL;
```

```
int packageCount = 0;
int option;
do
{
   printf("\nPackage Delivery Optimization Tool\n");
   printf("1. Add Package\n");
   printf("2. Display Package Details\n");
   printf("3. Exit\n");
   printf("Enter the option: ");
   scanf("%d", &option);
   switch (option)
   {
      case 1: addPackage(&packages, &packageCount);
            break;
      case 2: displayPackageDetails(packages, packageCount);
            break;
      case 3: for (int i = 0; i < packageCount; i++)
            {
               free((int *)packages[i].packageID);
               free(packages[i].deliveryDetails.deliveryMethod);
            }
            free(packages);
            printf("Exiting the tool\n");
            break;
      default: printf("Invalid option\n");
   }
} while (option != 3);
```

```
    return 0;
}

void addPackage(struct Package **packages, int *packageCount)
{
    int tempPackageID;
    struct Package newPackage;
    printf("Enter Package ID: ");
    scanf("%d", &tempPackageID);
    int *packageIDPtr = (int *)malloc(sizeof(int));
    *packageIDPtr = tempPackageID;
    newPackage.packageID = packageIDPtr;
    printf("Enter Package Weight: ");
    scanf("%f", &newPackage.weight);
    printf("Enter Package Destination: ");
    scanf(" %[^\n]", newPackage.destination);
    newPackage.deliveryDetails.deliveryMethod = (char *)malloc
                                                    (20 * sizeof(char));
    printf("Enter Delivery Method (standard or express): ");
    scanf(" %[^\n]", newPackage.deliveryDetails.deliveryMethod);
    *packages = (struct Package *)realloc(*packages, (*packageCount + 1)
                                                    * sizeof(struct Package));
    (*packages)[*packageCount] = newPackage;
    (*packageCount)++;
    printf("Package added successfully\n");
}
```

```
void displayPackageDetails(struct Package *packages, int packageCount)
{
   if (packageCount == 0)
   {
      printf("No packages added yet\n");
      return;
   }
   printf("\nPackage Delivery Details\n");
   for (int i = 0; i < packageCount; i++)
   {
      printf("Package ID: %d\n", *(packages[i].packageID));
      printf("Weight: %.2f kg\n", packages[i].weight);
      printf("Destination: %s\n", packages[i].destination);
      printf("Delivery Method: %s\n",
                        packages[i].deliveryDetails.deliveryMethod);
   }
}
```

9. Logistics Data Analytics System

Description:
Create a logistics data analytics system using structures for analytics records and unions for different metrics. Use const pointers to ensure data integrity and double pointers for managing dynamic analytics data.

Specifications:

- Structure: Analytics records (timestamp, metric).
- Union: Metrics (speed, efficiency).
- const Pointer: Analytics data.

- Double Pointers: Dynamic data storage.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct AnalyticsRecord
{
   const int *timestamp;
   union
   {
      float speed;
      float efficiency;
   } metric;
   int type; // 1 for speed, 0 for efficiency
};

void addAnalyticsRecord(struct AnalyticsRecord **records, int *recordCount);
void displayAnalyticsDetails(struct AnalyticsRecord *records, int recordCount);

int main()
{
   struct AnalyticsRecord *records = NULL;
   int recordCount = 0, option;
   do
   {
      printf("\nLogistics Data Analytics System\n");
```

```
        printf("1. Add Analytics Record\n");
        printf("2. Display Analytics Details\n");
        printf("3. Exit\n");
        printf("Enter the option: ");
        scanf("%d", &option);
        switch (option)
        {
            case 1: addAnalyticsRecord(&records, &recordCount);
                    break;
            case 2: displayAnalyticsDetails(records, recordCount);
                    break;
            case 3: for (int i = 0; i < recordCount; i++)
                        free((int *)records[i].timestamp);
                    free(records);
                    printf("Exiting the system\n");
                    break;
            default: printf("Invalid option\n");
        }
    } while (option != 3);
    return 0;
}

void addAnalyticsRecord(struct AnalyticsRecord **records, int *recordCount)
{
    int tempTimestamp;
    struct AnalyticsRecord newRecord;
    printf("Enter Timestamp: ");
```

```
    scanf("%d", &tempTimestamp);
    int *timestampPtr = (int *)malloc(sizeof(int));
    *timestampPtr = tempTimestamp;
    newRecord.timestamp = timestampPtr;
    printf("Enter Metric Type (1 for Speed, 0 for Efficiency): ");
    scanf("%d", &newRecord.type);
    if (newRecord.type)
    {
        printf("Enter Speed: ");
        scanf(" %f", &newRecord.metric.speed);
    }
    else
    {
        printf("Enter Efficiency: ");
        scanf(" %f", &newRecord.metric.efficiency);
    }
    *records = (struct AnalyticsRecord *)realloc(*records, (*recordCount + 1) *
                                                 sizeof(struct AnalyticsRecord));
    (*records)[*recordCount] = newRecord;
    (*recordCount)++;
    printf("Analytics record added successfully\n");
}

void displayAnalyticsDetails(struct AnalyticsRecord *records, int recordCount)
{
    if (recordCount == 0)
    {
```

```
        printf("No analytics records\n");
        return;
    }
    printf("\nLogistics Data Analytics Details\n");
    for (int i = 0; i < recordCount; i++)
    {
        printf("Timestamp: %d\n", *(records[i].timestamp));
        if (records[i].type)
            printf("Metric (Speed): %.2f km/h\n", records[i].metric.speed);
        else
            printf("Metric (Efficiency): %.2f\n", records[i].metric.efficiency);
    }
}
```

10. Transportation Schedule Management

Description:
Implement a transportation schedule management system using structures for schedule details and unions for transport types. Use const pointers for schedule IDs and double pointers for dynamic schedule lists.

Specifications:

- Structure: Schedule details (ID, start time, end time).
- Union: Transport types (bus, truck).
- const Pointer: Schedule IDs.
- Double Pointers: Dynamic schedule handling.

```
#include <stdio.h>
#include <stdlib.h>
```

```
#include <string.h>

struct Schedule
{
    const int *scheduleID;
    char startTime[10];
    char endTime[10];
    union
    {
        char bus[20];
        char truck[20];
    } transportType;
    int type; // 1 for bus, 0 for truck
};

void addSchedule(struct Schedule **schedules, int *scheduleCount);
void displayScheduleDetails(struct Schedule *schedules, int scheduleCount);

int main()
{
    struct Schedule *schedules = NULL;
    int scheduleCount = 0, option;
    do
    {
        printf("\nTransportation Schedule Management\n");
        printf("1. Add Schedule\n");
        printf("2. Display Schedule Details\n");
```

```
        printf("3. Exit\n");
        printf("Enter the option: ");
        scanf("%d", &option);
        switch (option)
        {
            case 1: addSchedule(&schedules, &scheduleCount); break;
            case 2: displayScheduleDetails(schedules, scheduleCount); break;
            case 3: for (int i = 0; i < scheduleCount; i++)
                        free((int *)schedules[i].scheduleID);
                    free(schedules);
                    printf("Exiting the management\n");
                    break;
            default: printf("Invalid option\n");
        }
    } while (option != 3);
    return 0;
}

void addSchedule(struct Schedule **schedules, int *scheduleCount)
{
    int tempScheduleID;
    struct Schedule newSchedule;
    printf("Enter Schedule ID: ");
    scanf("%d", &tempScheduleID);
    int *scheduleIDPtr = (int *)malloc(sizeof(int));
    *scheduleIDPtr = tempScheduleID;
    newSchedule.scheduleID = scheduleIDPtr;
```

```
    printf("Enter Start Time: ");
    scanf("%s", newSchedule.startTime);
    printf("Enter End Time: ");
    scanf("%s", newSchedule.endTime);
    printf("Enter Transport Type (1 for Bus, 0 for Truck): ");
    scanf("%d", &newSchedule.type);
    if (newSchedule.type)
    {
        printf("Enter Bus Details: ");
        scanf(" %[^\n]", newSchedule.transportType.bus);
    }
    else
    {
        printf("Enter Truck Details: ");
        scanf(" %[^\n]", newSchedule.transportType.truck);
    }
    *schedules = (struct Schedule *)realloc(*schedules, (*scheduleCount + 1) *
                                                    sizeof(struct Schedule));
    (*schedules)[*scheduleCount] = newSchedule;
    (*scheduleCount)++;
    printf("Schedule added successfully\n");
}

void displayScheduleDetails(struct Schedule *schedules, int scheduleCount)
{
    if (scheduleCount == 0)
    {
```

```
        printf("No schedules available\n");
        return;
    }
    printf("\nTransportation Schedule Details\n");
    for (int i = 0; i < scheduleCount; i++)
    {
        printf("Schedule ID: %d\n", *(schedules[i].scheduleID));
        printf("Start Time: %s\n", schedules[i].startTime);
        printf("End Time: %s\n", schedules[i].endTime);
        if (schedules[i].type)
            printf("Transport Type: Bus - %s\n", schedules[i].transportType.bus);
        else
            printf("Transport Type: Truck - %s\n", schedules[i].transportType.truck);
    }
}
```

11. Dynamic Supply Chain Modeling

Description:
Develop a dynamic supply chain modeling tool using structures for supplier and customer details, and unions for transaction types. Use const pointers for transaction IDs and double pointers for dynamic relationship management.

Specifications:

- Structure: Supplier/customer details (ID, name).
- Union: Transaction types (purchase, return).
- const Pointer: Transaction IDs.
- Double Pointers: Dynamic supply chain modeling.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct SupplyChainTransaction
{
   const int *transactionID;
   char supplierName[50];
   char customerName[50];
   union
   {
      char purchase[100];
      char returnDetails[100];
   } transactionType;
   int type; // 1 for purchase, 0 for return
};

void addTransaction(struct SupplyChainTransaction **transactions,
                                                int *transactionCount);
void displayTransactionDetails(struct SupplyChainTransaction *transactions,
                                                int transactionCount);

int main()
{
   struct SupplyChainTransaction *transactions = NULL;
   int transactionCount = 0, option;
   do
```

```
    {
        printf("\nDynamic Supply Chain Modeling\n");
        printf("1. Add Transaction\n");
        printf("2. Display Transaction Details\n");
        printf("3. Exit\n");
        printf("Enter the option: ");
        scanf("%d", &option);
        switch (option)
        {
            case 1: addTransaction(&transactions, &transactionCount); break;
            case 2: displayTransactionDetails(transactions, transactionCount);  break;
            case 3: for (int i = 0; i < transactionCount; i++)
                        free((int *)transactions[i].transactionID);
                    free(transactions);
                    printf("Exiting the supply chain modeling tool\n");
                    break;
            default: printf("Invalid option\n");
        }
    } while (option != 3);
    return 0;
}

void addTransaction(struct SupplyChainTransaction **transactions,
                                                        int *transactionCount)
{
    int tempTransactionID;
    struct SupplyChainTransaction newTransaction;
```

```
    printf("Enter Transaction ID: ");
    scanf("%d", &tempTransactionID);
    int *transactionIDPtr = (int *)malloc(sizeof(int));
    *transactionIDPtr = tempTransactionID;
    newTransaction.transactionID = transactionIDPtr;
    printf("Enter Supplier Name: ");
    scanf(" %[^\n]", newTransaction.supplierName);
    printf("Enter Customer Name: ");
    scanf(" %[^\n]", newTransaction.customerName);
    printf("Enter Transaction Type (1 for Purchase, 0 for Return): ");
    scanf("%d", &newTransaction.type);
    if (newTransaction.type)
    {
        printf("Enter Purchase Details: ");
        scanf(" %[^\n]", newTransaction.transactionType.purchase);
    }
    else
    {
        printf("Enter Return Details: ");
        scanf(" %[^\n]", newTransaction.transactionType.returnDetails);
    }
    *transactions = (struct SupplyChainTransaction *)realloc(*transactions,
                (*transactionCount + 1) * sizeof(struct SupplyChainTransaction));
    (*transactions)[*transactionCount] = newTransaction;
    (*transactionCount)++;
    printf("Transaction added successfully\n");
}
```

```
void displayTransactionDetails(struct SupplyChainTransaction *transactions,
                                        int transactionCount)
{
   if (transactionCount == 0)
   {
      printf("No transactions available\n");
      return;
   }
   printf("\nTransaction Details\n");
   for (int i = 0; i < transactionCount; i++)
   {
      printf("Transaction ID: %d\n", *(transactions[i].transactionID));
      printf("Supplier: %s\n", transactions[i].supplierName);
      printf("Customer: %s\n", transactions[i].customerName);
      if (transactions[i].type)
         printf("Transaction Type: Purchase - %s\n",
                  transactions[i].transactionType.purchase);
      else
         printf("Transaction Type: Return - %s\n",
                  transactions[i].transactionType.returnDetails);
   }
}
```

12. Freight Cost Calculation System

Description:
Create a freight cost calculation system using structures for cost components and

unions for different pricing models. Use const pointers for fixed cost parameters and double pointers for dynamically allocated cost records.

Specifications:

- Structure: Cost components (ID, base cost).
- Union: Pricing models (fixed, variable).
- const Pointer: Cost parameters.
- Double Pointers: Dynamic cost management.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct CostComponent
{
   const int *costID;
   float baseCost;
   union
   {
      float fixedCost;
      float variableRate;
   } pricingModel;
   int type; // 1 for fixed cost, 0 for variable rate
};

void addCostRecord(struct CostComponent **costs, int *costCount);
void displayCostRecords(struct CostComponent *costs, int costCount);

int main()
```

```
{
    struct CostComponent *costs = NULL;
    int costCount = 0, option;
    do
    {
        printf("\nFreight Cost Calculation System\n");
        printf("1. Add Cost Record\n");
        printf("2. Display Cost Records\n");
        printf("3. Exit\n");
        printf("Enter the option: ");
        scanf("%d", &option);
        switch (option)
        {
        case 1: addCostRecord(&costs, &costCount); break;
        case 2: displayCostRecords(costs, costCount); break;
        case 3: for (int i = 0; i < costCount; i++)
                    free((int *)costs[i].costID);
                free(costs);
                printf("Exiting the system\n");
                break;
        default: printf("Invalid option\n");
        }
    } while (option != 3);
    return 0;
}

void addCostRecord(struct CostComponent **costs, int *costCount)
```

```
{
    int tempCostID;
    struct CostComponent newCost;
    printf("Enter Cost ID: ");
    scanf("%d", &tempCostID);
    int *costIDPtr = (int *)malloc(sizeof(int));
    *costIDPtr = tempCostID;
    newCost.costID = costIDPtr;
    printf("Enter Base Cost: ");
    scanf("%f", &newCost.baseCost);
    printf("Select Pricing Model (1 for Fixed Cost, 0 for Variable Rate): ");
    scanf("%d", &newCost.type);
    if (newCost.type)
    {
        printf("Enter Fixed Cost: ");
        scanf("%f", &newCost.pricingModel.fixedCost);
    }
    else {
        printf("Enter Variable Rate (per unit): ");
        scanf("%f", &newCost.pricingModel.variableRate);
    }
    *costs = (struct CostComponent *)realloc(*costs, (*costCount + 1) *
                                             sizeof(struct CostComponent));
    (*costs)[*costCount] = newCost;
    (*costCount)++;
    printf("Cost record added successfully\n");
}
```

```
void displayCostRecords(struct CostComponent *costs, int costCount)
{
   if (costCount == 0)
   {
      printf("No cost records available\n");
      return;
   }
   printf("\nFreight Cost Records\n");
   for (int i = 0; i < costCount; i++)
   {
      printf("Cost ID: %d\n", *(costs[i].costID));
      printf("Base Cost: %.2f\n", costs[i].baseCost);
      if (costs[i].type)
         printf("Pricing Model: Fixed Cost - %.2f\n",
                              costs[i].pricingModel.fixedCost);
      else
         printf("Pricing Model: Variable Rate - %.2f per unit\n",
                                    costs[i].pricingModel.variableRate);
   }
}
```

13. Vehicle Load Balancing System

Description:
Design a vehicle load balancing system using structures for load details and unions for load types. Use const pointers for load identifiers and double pointers for managing dynamic load distribution.

Specifications:

- Structure: Load details (ID, weight, destination).
- Union: Load types (bulk, container).
- const Pointer: Load IDs.
- Double Pointers: Dynamic load handling.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct LoadDetails
{
   const int *loadID;
   float weight;
   char destination[50];
   union
   {
      char bulk[50];
      char container[50];
   } loadType;
   int type; // 1 for bulk, 0 for container
};

void addLoadRecord(struct LoadDetails **loads, int *loadCount);
void displayLoadRecords(struct LoadDetails *loads, int loadCount);

int main()
{
```

```
    struct LoadDetails *loads = NULL;
    int loadCount = 0, option;
    do
    {
        printf("\nVehicle Load Balancing System\n");
        printf("1. Add Load Record\n");
        printf("2. Display Load Records\n");
        printf("3. Exit\n");
        printf("Enter the option: ");
        scanf("%d", &option);
        switch (option)
        {
            case 1: addLoadRecord(&loads, &loadCount); break;
            case 2: displayLoadRecords(loads, loadCount); break;
            case 3: for (int i = 0; i < loadCount; i++)
                        free((int *)loads[i].loadID);
                    free(loads);
                    printf("Exiting the system\n");
                    break;
            default: printf("Invalid option\n");
        }
    } while (option != 3);
    return 0;
}

void addLoadRecord(struct LoadDetails **loads, int *loadCount)
{
```

```
int tempLoadID;
struct LoadDetails newLoad;
printf("Enter Load ID: ");
scanf("%d", &tempLoadID);
int *loadIDPtr = (int *)malloc(sizeof(int));
*loadIDPtr = tempLoadID;
newLoad.loadID = loadIDPtr;
printf("Enter Weight: ");
scanf("%f", &newLoad.weight);
printf("Enter Destination: ");
scanf(" %[^\n]", newLoad.destination);
printf("Select Load Type (1 for Bulk, 0 for Container): ");
scanf("%d", &newLoad.type);
if (newLoad.type)
{
   printf("Enter Bulk Load Description: ");
   scanf(" %[^\n]", newLoad.loadType.bulk);
}
else
{
   printf("Enter Container Load Description: ");
   scanf(" %[^\n]", newLoad.loadType.container);
}
*loads = (struct LoadDetails *)realloc(*loads, (*loadCount + 1) *
                                        sizeof(struct LoadDetails));
(*loads)[*loadCount] = newLoad;
(*loadCount)++;
```

```
    printf("Load record added successfully\n");
}

void displayLoadRecords(struct LoadDetails *loads, int loadCount)
{
    if (loadCount == 0)
    {
        printf("No load records available\n");
        return;
    }
    printf("\nLoad Records\n");
    for (int i = 0; i < loadCount; i++)
    {
        printf("Load ID: %d\n", *(loads[i].loadID));
        printf("Weight: %.2f kg\n", loads[i].weight);
        printf("Destination: %s\n", loads[i].destination);
        if (loads[i].type)
            printf("Load Type: Bulk - %s\n", loads[i].loadType.bulk);
        else
            printf("Load Type: Container - %s\n", loads[i].loadType.container);
    }
}
```

14. Intermodal Transport Management System

Description:
Implement an intermodal transport management system using structures for

transport details and unions for transport modes. Use const pointers for transport identifiers and double pointers for dynamic transport route management.

Specifications:

- Structure: Transport details (ID, origin, destination).
- Union: Transport modes (rail, road).
- const Pointer: Transport IDs.
- Double Pointers: Dynamic transport management.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct TransportDetails
{
   const int *transportID;
   char origin[50];
   char destination[50];
   union
   {
      char railDetails[50];
      char roadDetails[50];
   } transportMode;
   int type; // 1 for rail, 0 for road
};

void addTransportRecord(struct TransportDetails **transports,
                                                    int *transportCount);
void displayTransportRecords(struct TransportDetails *transports,
```

```
                                                    int transportCount);

int main()
{
   struct TransportDetails *transports = NULL;
   int transportCount = 0, option;
   do{
      printf("\nIntermodal Transport Management System\n");
      printf("1. Add Transport Record\n");
      printf("2. Display Transport Records\n");
      printf("3. Exit\n");
      printf("Enter the option: ");
      scanf("%d", &option);
      switch (option)
      {
         case 1: addTransportRecord(&transports, &transportCount); break;
         case 2: displayTransportRecords(transports, transportCount); break;
         case 3: for (int i = 0; i < transportCount; i++)
                        free((int *)transports[i].transportID);
                 free(transports);
                 printf("Exiting the system\n");
                 break;
      default: printf("Invalid option\n");
      }
   } while (option != 3);
   return 0;
}
```

```
void addTransportRecord(struct TransportDetails **transports,
                                                    int *transportCount)
{
   int tempTransportID;
   struct TransportDetails newTransport;
   printf("Enter Transport ID: ");
   scanf("%d", &tempTransportID);
   int *transportIDPtr = (int *)malloc(sizeof(int));
   *transportIDPtr = tempTransportID;
   newTransport.transportID = transportIDPtr;
   printf("Enter Origin: ");
   scanf(" %[^\n]", newTransport.origin);
   printf("Enter Destination: ");
   scanf(" %[^\n]", newTransport.destination);
   printf("Select Transport Mode (1 for Rail, 0 for Road): ");
   scanf("%d", &newTransport.type);
   if (newTransport.type)
   {
      printf("Enter Rail Transport Details: ");
      scanf(" %[^\n]", newTransport.transportMode.railDetails);
   }
   else
   {
      printf("Enter Road Transport Details: ");
      scanf(" %[^\n]", newTransport.transportMode.roadDetails);
   }
   *transports = (struct TransportDetails *)realloc(*transports,
```

```
                    (*transportCount + 1) * sizeof(struct TransportDetails));
    (*transports)[*transportCount] = newTransport;
    (*transportCount)++;
    printf("Transport record added successfully\n");
}

void displayTransportRecords(struct TransportDetails *transports,
                                                    int transportCount)
{
    if (transportCount == 0){
        printf("No transport records\n");
        return;
    }
    printf("\nTransport Records\n");
    for (int i = 0; i < transportCount; i++)
    {
        printf("Transport ID: %d\n", *(transports[i].transportID));
        printf("Origin: %s\n", transports[i].origin);
        printf("Destination: %s\n", transports[i].destination);
        if (transports[i].type)
            printf("Transport Mode: Rail - %s\n",
                            transports[i].transportMode.railDetails);
        else
            printf("Transport Mode: Road - %s\n",
                            transports[i].transportMode.roadDetails);
    }
}
```

15. Logistics Performance Monitoring

Description:
Develop a logistics performance monitoring system using structures for performance metrics and unions for different performance aspects. Use const pointers for metric identifiers and double pointers for managing dynamic performance records.

Specifications:

- Structure: Performance metrics (ID, value).
- Union: Performance aspects (time, cost).
- const Pointer: Metric IDs.
- Double Pointers: Dynamic performance tracking.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct PerformanceMetrics
{
   const int *metricID;
   float value;
   union
   {
      float time;
      float cost;
   } performanceAspect;
   int type; // 1 for time, 0 for cost
};

void addPerformanceRecord(struct PerformanceMetrics **records,
```

```
                                                    int *recordCount);
void displayPerformanceRecords(struct PerformanceMetrics *records,
                                                    int recordCount);

int main()
{
   struct PerformanceMetrics *records = NULL;
   int recordCount = 0, option;
   do
   {
      printf("\nLogistics Performance Monitoring System\n");
      printf("1. Add Performance Record\n");
      printf("2. Display Performance Records\n");
      printf("3. Exit\n");
      printf("Enter the option: ");
      scanf("%d", &option);
      switch (option)
      {
         case 1: addPerformanceRecord(&records, &recordCount);
                 break;
         case 2: displayPerformanceRecords(records, recordCount);
                 break;
         case 3: for (int i = 0; i < recordCount; i++)
                    free((int *)records[i].metricID);
                 free(records);
                 printf("Exiting\n");
                 break;
```

```
            default: printf("Invalid option\n");
        }
    } while (option != 3);
    return 0;
}

void addPerformanceRecord(struct PerformanceMetrics **records,
                                                    int *recordCount)
{
    int tempMetricID;
    struct PerformanceMetrics newRecord;
    printf("Enter Metric ID: ");
    scanf("%d", &tempMetricID);
    int *metricIDPtr = (int *)malloc(sizeof(int));
    *metricIDPtr = tempMetricID;
    newRecord.metricID = metricIDPtr;
    printf("Enter Metric Value: ");
    scanf("%f", &newRecord.value);
    printf("Select Performance Aspect (1 for Time, 0 for Cost): ");
    scanf("%d", &newRecord.type);
    if (newRecord.type)
    {
        printf("Enter Time: ");
        scanf("%f", &newRecord.performanceAspect.time);
    }
    else
    {
```

```
        printf("Enter Cost: ");
        scanf("%f", &newRecord.performanceAspect.cost);
    }
    *records = (struct PerformanceMetrics *)realloc(*records, (*recordCount + 1)
                                                * sizeof(struct PerformanceMetrics));
    (*records)[*recordCount] = newRecord;
    (*recordCount)++;
    printf("Performance record added successfully\n");
}

void displayPerformanceRecords(struct PerformanceMetrics *records,
                                                                int recordCount)
{
    if (recordCount == 0)
    {
        printf("No performance records available\n");
        return;
    }
    printf("\nPerformance Records\n");
    for (int i = 0; i < recordCount; i++)
    {
        printf("Metric ID: %d\n", *(records[i].metricID));
        printf("Metric Value: %.2f\n", records[i].value);
        if (records[i].type)
            printf("Performance Aspect: Time - %.2f \n",
                                records[i].performanceAspect.time);
        else
```

```
        printf("Performance Aspect: Cost - %.2f\n",
                                        records[i].performanceAspect.cost);
    }
}
```

16. Warehouse Robotics Coordination

Description:
Create a system to coordinate warehouse robotics using structures for robot details and unions for task types. Use const pointers for robot identifiers and double pointers for managing dynamic task allocations.

Specifications:

- Structure: Robot details (ID, type, status).
- Union: Task types (picking, sorting).
- const Pointer: Robot IDs.
- Double Pointers: Dynamic task management.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Robot
{
    const int *robotID;
    char status[20];
    union
    {
        char picking[100];
```

```
        char sorting[100];
    } taskType;
    int type; // 1 for picking, 0 for sorting
};

void addRobot(struct Robot **robots, int *robotCount);
void displayRobotDetails(struct Robot *robots, int robotCount);

int main()
{
    struct Robot *robots = NULL;
    int robotCount = 0, option;
    do
    {
        printf("\nWarehouse Robotics Coordination System\n");
        printf("1. Add Robot\n");
        printf("2. Display Robot Details\n");
        printf("3. Exit\n");
        printf("Enter the option: ");
        scanf("%d", &option);
        switch (option)
        {
            case 1: addRobot(&robots, &robotCount); break;
            case 2: displayRobotDetails(robots, robotCount); break;
            case 3: for (int i = 0; i < robotCount; i++)
                        free((int *)robots[i].robotID);
                    free(robots);
```

```
                printf("Exiting the system\n");
                break;
            default: printf("Invalid option\n");
        }
    } while (option != 3);
    return 0;
}

void addRobot(struct Robot **robots, int *robotCount)
{
    int tempRobotID;
    struct Robot newRobot;
    printf("Enter Robot ID: ");
    scanf("%d", &tempRobotID);
    int *robotIDPtr = (int *)malloc(sizeof(int));
    *robotIDPtr = tempRobotID;
    newRobot.robotID = robotIDPtr;
    printf("Enter Robot Status (Idle, Working): ");
    scanf(" %[^\n]", newRobot.status);
    printf("Select Task Types (1 for Picking, 0 for Sorting): ");
    scanf(" %d", &newRobot.type);
    if (newRobot.type)
    {
        printf("Enter Task: ");
        scanf(" %[^\n]", newRobot.taskType.picking);
    }
    else{
```

```
        printf("Enter Task: ");
        scanf(" %[^\n]", newRobot.taskType.sorting);
    }
    *robots = (struct Robot *)realloc(*robots, (*robotCount + 1) *
                                                    sizeof(struct Robot));
    (*robots)[*robotCount] = newRobot;
    (*robotCount)++;
    printf("Robot added successfully\n");
}

void displayRobotDetails(struct Robot *robots, int robotCount)
{
    if (robotCount == 0) {
        printf("No robots available\n");
        return;
    }
    printf("\nRobot Details\n");
    for (int i = 0; i < robotCount; i++)
    {
        printf("Robot ID: %d\n", *(robots[i].robotID));
        printf("Robot Status: %s\n", robots[i].status);
        if (robots[i].type)
            printf("Task: Picking - %s\n", robots[i].taskType.picking);
        else
            printf("Task: Sorting - %s\n", robots[i].taskType.sorting);
    }
}
```

17. Customer Feedback Analysis System

Description:
Design a system to analyze customer feedback using structures for feedback details and unions for feedback types. Use const pointers for feedback IDs and double pointers for dynamically managing feedback data.

Specifications:

- Structure: Feedback details (ID, content).
- Union: Feedback types (positive, negative).
- const Pointer: Feedback IDs.
- Double Pointers: Dynamic feedback management.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Feedback
{
   const int *feedbackID;
   char content[200];
   union
   {
      char positive[200];
      char negative[200];
   } feedbackType;
   int type; // 1 for positive, 0 for negative
};

void addFeedback(struct Feedback **feedbacks, int *feedbackCount);
```

```
void displayFeedbackDetails(struct Feedback *feedbacks, int feedbackCount);

int main()
{
   struct Feedback *feedbacks = NULL;
   int feedbackCount = 0, option;
   do
   {
      printf("\nCustomer Feedback Analysis System\n");
      printf("1. Add Feedback\n");
      printf("2. Display Feedback Details\n");
      printf("3. Exit\n");
      printf("Enter the option: ");
      scanf("%d", &option);
      switch (option)
      {
         case 1: addFeedback(&feedbacks, &feedbackCount); break;
         case 2: displayFeedbackDetails(feedbacks, feedbackCount); break;
         case 3: for (int i = 0; i < feedbackCount; i++)
                    free((int *)feedbacks[i].feedbackID);
                 free(feedbacks);
                 printf("Exiting the system\n");
                 break;
         default: printf("Invalid option\n");
      }
   } while (option != 3);
   return 0;
```

```
}

void addFeedback(struct Feedback **feedbacks, int *feedbackCount)
{
   int tempFeedbackID;
   struct Feedback newFeedback;
   printf("Enter Feedback ID: ");
   scanf("%d", &tempFeedbackID);
   int *feedbackIDPtr = (int *)malloc(sizeof(int));
   *feedbackIDPtr = tempFeedbackID;
   newFeedback.feedbackID = feedbackIDPtr;
   printf("Enter Feedback Content: ");
   scanf(" %[^\n]", newFeedback.content);
   printf("Select Feedback Type (1 for Positive, 0 for Negative): ");
   scanf(" %d", &newFeedback.type);
   if (newFeedback.type)
   {
      printf("Enter Positive Feedback: ");
      scanf(" %[^\n]", newFeedback.feedbackType.positive);
   }
   else
   {
      printf("Enter Negative Feedback: ");
      scanf(" %[^\n]", newFeedback.feedbackType.negative);
   }
   *feedbacks = (struct Feedback *)realloc(*feedbacks, (*feedbackCount + 1) *
                                                   sizeof(struct Feedback));
```

```
    (*feedbacks)[*feedbackCount] = newFeedback;
    (*feedbackCount)++;
    printf("Feedback added successfully\n");
}

void displayFeedbackDetails(struct Feedback *feedbacks, int feedbackCount)
{
    if (feedbackCount == 0)
    {
        printf("No feedback\n");
        return;
    }
    printf("\nFeedback Details\n");
    for (int i = 0; i < feedbackCount; i++)
    {
        printf("Feedback ID: %d\n", *(feedbacks[i].feedbackID));
        printf("Content: %s\n", feedbacks[i].content);
        if (feedbacks[i].type)
            printf("Type: Positive - %s\n", feedbacks[i].feedbackType.positive);
        else
            printf("Type: Negative - %s\n", feedbacks[i].feedbackType.negative);
    }
}
```

18. Real-Time Fleet Coordination

Description:
Implement a real-time fleet coordination system using structures for fleet details

and unions for coordination types. Use const pointers for fleet IDs and double pointers for managing dynamic coordination data.

Specifications:

- Structure: Fleet details (ID, location, status).
- Union: Coordination types (dispatch, reroute).
- const Pointer: Fleet IDs.
- Double Pointers: Dynamic coordination.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Fleet
{
   const int *fleetID;
   char location[100];
   char status[50];
   union
   {
      char dispatchDetails[200];
      char rerouteDetails[200];
   } coordinationType;
   int type; // 1 for dispatch, 0 for reroute
};

void addFleet(struct Fleet **fleets, int *fleetCount);
void displayFleetDetails(struct Fleet *fleets, int fleetCount);
```

```
int main()
{
    struct Fleet *fleets = NULL;
    int fleetCount = 0, option;
    do
    {
        printf("\nReal-Time Fleet Coordination System\n");
        printf("1. Add Fleet\n");
        printf("2. Display Fleet Details\n");
        printf("3. Exit\n");
        printf("Enter the option: ");
        scanf("%d", &option);
        switch (option)
        {
            case 1: addFleet(&fleets, &fleetCount); break;
            case 2: displayFleetDetails(fleets, fleetCount); break;
            case 3: for (int i = 0; i < fleetCount; i++)
                        free((int *)fleets[i].fleetID);
                    free(fleets);
                    printf("Exiting the system\n");
                    break;
            default: printf("Invalid option\n");
        }
    } while (option != 3);
    return 0;
}
```

```
void addFleet(struct Fleet **fleets, int *fleetCount)
{
    int tempFleetID;
    struct Fleet newFleet;
    printf("Enter Fleet ID: ");
    scanf("%d", &tempFleetID);
    int *fleetIDPtr = (int *)malloc(sizeof(int));
    *fleetIDPtr = tempFleetID;
    newFleet.fleetID = fleetIDPtr;
    printf("Enter Fleet Location: ");
    scanf(" %[^\n]", newFleet.location);
    printf("Enter Fleet Status (Active/Inactive): ");
    scanf(" %[^\n]", newFleet.status);
    printf("Select Coordination Type (1 for Dispatch, 0 for Reroute): ");
    scanf("%d", &newFleet.type);
    if (newFleet.type)
    {
        printf("Enter Dispatch Details: ");
        scanf(" %[^\n]", newFleet.coordinationType.dispatchDetails);
    }
    else
    {
        printf("Enter Reroute Details: ");
        scanf(" %[^\n]", newFleet.coordinationType.rerouteDetails);
    }
    *fleets = (struct Fleet *)realloc(*fleets, (*fleetCount + 1) * sizeof(struct Fleet));
    (*fleets)[*fleetCount] = newFleet;
```

```
    (*fleetCount)++;
    printf("Fleet added successfully\n");
}

void displayFleetDetails(struct Fleet *fleets, int fleetCount)
{
    if (fleetCount == 0)
    {
        printf("No fleets available\n");
        return;
    }
    printf("\nFleet Details\n");
    for (int i = 0; i < fleetCount; i++)
    {
        printf("Fleet ID: %d\n", *(fleets[i].fleetID));
        printf("Location: %s\n", fleets[i].location);
        printf("Status: %s\n", fleets[i].status);
        if (fleets[i].type)
            printf("Coordination: Dispatch - %s\n",
                        fleets[i].coordinationType.dispatchDetails);
        else
            printf("Coordination: Reroute - %s\n",
                        fleets[i].coordinationType.rerouteDetails);
    }
}
```

19. Logistics Security Management System

Description:
Develop a security management system for logistics using structures for security events and unions for event types. Use const pointers for event identifiers and double pointers for managing dynamic security data.

Specifications:

- Structure: Security events (ID, description).
- Union: Event types (breach, resolved).
- const Pointer: Event IDs.
- Double Pointers: Dynamic security event handling.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct SecurityEvent
{
   const int *eventID;
   char description[200];
   union
   {
      char breachDetails[200];
      char resolvedDetails[200];
   } eventType;
   int type; // 1 for breach, 0 for resolved
};

void addSecurityEvent(struct SecurityEvent **events, int *eventCount);
```

```
void displaySecurityEvents(struct SecurityEvent *events, int eventCount);

int main()
{
   struct SecurityEvent *events = NULL;
   int eventCount = 0, option;
   do
   {
      printf("\nLogistics Security Management System\n");
      printf("1. Add Security Event\n");
      printf("2. Display Security Events\n");
      printf("3. Exit\n");
      printf("Enter the option: ");
      scanf("%d", &option);
      switch (option)
      {
         case 1: addSecurityEvent(&events, &eventCount); break;
         case 2: displaySecurityEvents(events, eventCount); break;
         case 3: for (int i = 0; i < eventCount; i++)
                    free((int *)events[i].eventID);
                 free(events);
                 printf("Exiting the system\n");
                 break;
         default:
            printf("Invalid option\n");
      }
   } while (option != 3);
```

```
    return 0;
}

void addSecurityEvent(struct SecurityEvent **events, int *eventCount)
{
    int tempEventID;
    struct SecurityEvent newEvent;
    printf("Enter Event ID: ");
    scanf("%d", &tempEventID);
    int *eventIDPtr = (int *)malloc(sizeof(int));
    *eventIDPtr = tempEventID;
    newEvent.eventID = eventIDPtr;
    printf("Enter Event Description: ");
    scanf(" %[^\n]", newEvent.description);
    printf("Select Event Type (1 for Breach, 0 for Resolved): ");
    scanf("%d", &newEvent.type);
    if (newEvent.type)
    {
        printf("Enter Breach Details: ");
        scanf(" %[^\n]", newEvent.eventType.breachDetails);
    }
    else
    {
        printf("Enter Resolved Details: ");
        scanf(" %[^\n]", newEvent.eventType.resolvedDetails);
    }
    *events = (struct SecurityEvent *)realloc(*events, (*eventCount + 1) *
```

```
                        sizeof(struct SecurityEvent));
    (*events)[*eventCount] = newEvent;
    (*eventCount)++;
    printf("Security event added successfully\n");
}

void displaySecurityEvents(struct SecurityEvent *events, int eventCount)
{
    if (eventCount == 0)
    {
        printf("No security events available\n");
        return;
    }
    printf("\nSecurity Event Details\n");
    for (int i = 0; i < eventCount; i++)
    {
        printf("Event ID: %d\n", *(events[i].eventID));
        printf("Description: %s\n", events[i].description);
        if (events[i].type)
            printf("Event Type: Breach - %s\n", events[i].eventType.breachDetails);
        else
            printf("Event Type: Resolved - %s\n",
                                        events[i].eventType.resolvedDetails);
    }
}
```

20. Automated Billing System for Logistics

Description:
Create an automated billing system using structures for billing details and unions for payment methods. Use const pointers for bill IDs and double pointers for dynamically managing billing records.

Specifications:

- Structure: Billing details (ID, amount, date).
- Union: Payment methods (bank transfer, cash).
- const Pointer: Bill IDs.
- Double Pointers: Dynamic billing management.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Billing
{
   const int *billID;
   float amount;
   char date[20];
   union
   {
      char bankTransferDetails[100];
      char cashDetails[100];
   } paymentMethod;
   int method; // 1 for bank transfer, 0 for cash
};
```

```
void addBillingRecord(struct Billing **bills, int *billCount);
void displayBillingRecords(struct Billing *bills, int billCount);

int main()
{
   struct Billing *bills = NULL;
   int billCount = 0, option;
   do{
      printf("\nAutomated Billing System\n");
      printf("1. Add Billing Record\n");
      printf("2. Display Billing Records\n");
      printf("3. Exit\n");
      printf("Enter the option: ");
      scanf("%d", &option);
      switch (option)
      {
         case 1: addBillingRecord(&bills, &billCount); break;
         case 2: displayBillingRecords(bills, billCount); break;
         case 3: for (int i = 0; i < billCount; i++)
                    free((int *)bills[i].billID);
                 free(bills);
                 printf("Exiting the system\n");
                 break;
         default:printf("Invalid option\n");
      }
   } while (option != 3);
   return 0;
```

```
}

void addBillingRecord(struct Billing **bills, int *billCount)
{
   int tempBillID;
   struct Billing newBill;
   printf("Enter Bill ID: ");
   scanf("%d", &tempBillID);
   int *billIDPtr = (int *)malloc(sizeof(int));
   *billIDPtr = tempBillID;
   newBill.billID = billIDPtr;
   printf("Enter Billing Amount: ");
   scanf("%f", &newBill.amount);
   printf("Enter Billing Date: ");
   scanf(" %[^\n]", newBill.date);
   printf("Select Payment Method (1 for Bank Transfer, 0 for Cash): ");
   scanf("%d", &newBill.method);
   if (newBill.method)
   {
      printf("Enter Bank Transfer Details: ");
      scanf(" %[^\n]", newBill.paymentMethod.bankTransferDetails);
   }
   else{
      printf("Enter Cash Details: ");
      scanf(" %[^\n]", newBill.paymentMethod.cashDetails);
   }
   *bills = (struct Billing *)realloc(*bills, (*billCount + 1) *
```

```
                                        sizeof(struct Billing));
    (*bills)[*billCount] = newBill;
    (*billCount)++;
    printf("Billing record added successfully\n");
}

void displayBillingRecords(struct Billing *bills, int billCount)
{
    if (billCount == 0)
    {
        printf("No billing records available\n");
        return;
    }
    printf("\nBilling Records\n");
    for (int i = 0; i < billCount; i++)
    {
        printf("Bill ID: %d\n", *(bills[i].billID));
        printf("Amount: %.2f\n", bills[i].amount);
        printf("Date: %s\n", bills[i].date);
        if (bills[i].method)
            printf("Payment Method: Bank Transfer - %s\n",
                        bills[i].paymentMethod.bankTransferDetails);
        else
            printf("Payment Method: Cash - %s\n",
                            bills[i].paymentMethod.cashDetails);
    }
}
```

1. Vessel Navigation System

Description:
Design a navigation system that tracks a vessel's current position and routes using structures and arrays. Use const pointers for immutable route coordinates and strings for location names. Double pointers handle dynamic route allocation.

Specifications:

- Structure: Route details (start, end, waypoints).
- Array: Stores multiple waypoints.
- Strings: Names of locations.
- const Pointers: Route coordinates.
- Double Pointers: Dynamic allocation of routes.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Route
{
   const char *startLocation;
   const char *endLocation;
   const char **waypoints;
   int waypointCount;
};

void addRoute(struct Route **routes, int *routeCount);
void displayRoutes(struct Route *routes, int routeCount);

int main()
```

```
{
    struct Route *routes = NULL;
    int routeCount = 0, option;
    do
    {
        printf("\nVessel Navigation System\n");
        printf("1. Add Route\n");
        printf("2. Display Routes\n");
        printf("3. Exit\n");
        printf("Enter the option: ");
        scanf("%d", &option);
        switch (option)
        {
            case 1: addRoute(&routes, &routeCount); break;
            case 2: displayRoutes(routes, routeCount); break;
            case 3: for (int i = 0; i < routeCount; i++)
                    {
                        free((char *)routes[i].startLocation);
                        free((char *)routes[i].endLocation);
                        for (int j = 0; j < routes[i].waypointCount; j++)
                            free((char *)routes[i].waypoints[j]);
                        free(routes[i].waypoints);
                    }
                    free(routes);
                    printf("Exiting the system\n");
                    break;
            default: printf("Invalid option\n");
```

```
        }
    } while (option != 3);
    return 0;
}

void addRoute(struct Route **routes, int *routeCount)
{
    char tempStart[100], tempEnd[100];
    int waypointCount;
    printf("Enter Start Location: ");
    scanf(" %[^\n]", tempStart);
    printf("Enter End Location: ");
    scanf(" %[^\n]", tempEnd);
    printf("Enter the number of waypoints: ");
    scanf("%d", &waypointCount);
    const char **waypoints = (const char **)malloc(waypointCount *
                                                   sizeof(char *));
    for (int i = 0; i < waypointCount; i++)
    {
        char tempWaypoint[100];
        printf("Enter Waypoint %d: ", i + 1);
        scanf(" %[^\n]", tempWaypoint);
        waypoints[i] = (char *)malloc((strlen(tempWaypoint) + 1) * sizeof(char));
        strcpy((char *)waypoints[i], tempWaypoint);
    }
    struct Route newRoute;
    newRoute.startLocation = (char *)malloc((strlen(tempStart) + 1) *
```

```
                                                    sizeof(char));
    strcpy((char *)newRoute.startLocation, tempStart);
    newRoute.endLocation = (char *)malloc((strlen(tempEnd) + 1) * sizeof(char));
    strcpy((char *)newRoute.endLocation, tempEnd);
    newRoute.waypoints = waypoints;
    newRoute.waypointCount = waypointCount;
    *routes = (struct Route *)realloc(*routes, (*routeCount + 1) *
                                                    sizeof(struct Route));
    (*routes)[*routeCount] = newRoute;
    (*routeCount)++;
    printf("Route added successfully\n");
}

void displayRoutes(struct Route *routes, int routeCount)
{
    if (routeCount == 0)
    {
        printf("No routes available\n");
        return;
    }
    printf("\nRoute Details\n");
    for (int i = 0; i < routeCount; i++)
    {
        printf("Route %d:\n", i + 1);
        printf("Start Location: %s\n", routes[i].startLocation);
        printf("End Location: %s\n", routes[i].endLocation);
        printf("Waypoints:\n");
```

```
        for (int j = 0; j < routes[i].waypointCount; j++)
            printf("    %d. %s\n", j + 1, routes[i].waypoints[j]);
    }
}
```

2. Fleet Management Software

Description:
Develop a system to manage multiple vessels in a fleet, using arrays for storing fleet data and structures for vessel details. Unions represent variable attributes like cargo type or passenger count.

Specifications:

- Structure: Vessel details (name, ID, type).
- Union: Cargo type or passenger count.
- Array: Fleet data.
- const Pointers: Immutable vessel IDs.
- Double Pointers: Manage dynamic fleet records.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Vessel
{
    const char *vesselID;
    char name[100];
    union CargoOrPassenger
    {
```

```
        char cargoType[50];
        int passengerCount;
    } details;
    int type; // 1 for cargo, 0 for passenger
};

void addVessel(struct Vessel **fleet, int *fleetCount);
void displayFleet(struct Vessel *fleet, int fleetCount);

int main()
{
    struct Vessel *fleet = NULL;
    int fleetCount = 0, option;
    do
    {
        printf("\nFleet Management Software\n");
        printf("1. Add Vessel\n");
        printf("2. Display Fleet\n");
        printf("3. Exit\n");
        printf("Enter the option: ");
        scanf("%d", &option);
        switch (option)
        {
            case 1: addVessel(&fleet, &fleetCount); break;
            case 2: displayFleet(fleet, fleetCount); break;
            case 3: for (int i = 0; i < fleetCount; i++)
                        free((char *)fleet[i].vesselID);
```

```
                free(fleet);
                printf("Exiting the software\n");
                break;
            default: printf("Invalid option\n");
        }
    } while (option != 3);
    return 0;
}

void addVessel(struct Vessel **fleet, int *fleetCount)
{
    char tempVesselID[50], tempName[100];
    int c;
    printf("Enter Vessel ID: ");
    scanf(" %[^\n]", tempVesselID);
    printf("Enter Vessel Name: ");
    scanf(" %[^\n]", tempName);
    printf("Select Type (1 for Cargo, 0 for Passenger): ");
    scanf("%d", &c);
    struct Vessel newVessel;
    newVessel.vesselID = (char *)malloc((strlen(tempVesselID) + 1) *
                                                    sizeof(char));
    strcpy((char *)newVessel.vesselID, tempVesselID);
    strcpy(newVessel.name, tempName);
    newVessel.type = c;
    if (c)
    {
```

```
        printf("Enter Cargo Type: ");
        scanf(" %[^\n]", newVessel.details.cargoType);
    }
    else
    {
        printf("Enter Number of Passengers: ");
        scanf("%d", &newVessel.details.passengerCount);
    }
    *fleet = (struct Vessel *)realloc(*fleet, (*fleetCount + 1) *
                                                    sizeof(struct Vessel));
    (*fleet)[*fleetCount] = newVessel;
    (*fleetCount)++;
    printf("Vessel added successfully\n");
}

void displayFleet(struct Vessel *fleet, int fleetCount)
{
    if (fleetCount == 0)
    {
        printf("No vessels available\n");
        return;
    }
    printf("\nFleet Details:\n");
    for (int i = 0; i < fleetCount; i++)
    {
        printf("Vessel %d:\n", i + 1);
        printf("Vessel ID: %s\n", fleet[i].vesselID);
```

```
        printf("Name: %s\n", fleet[i].name);
        if (fleet[i].type)
            printf("Cargo Type: %s\n", fleet[i].details.cargoType);
        else
            printf("Passenger Count: %d\n", fleet[i].details.passengerCount);
    }
}
```

3. Ship Maintenance Scheduler

Description:
Create a scheduler for ship maintenance tasks. Use structures to define tasks and arrays for schedules. Utilize double pointers for managing dynamic task lists.

Specifications:

- Structure: Maintenance task (ID, description, schedule).
- Array: Maintenance schedules.
- const Pointers: Read-only task IDs.
- Double Pointers: Dynamic task lists.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct MaintenanceTask
{
    const char *taskID;
    char description[200];
    char schedule[100];
```

```
};

void addTask(struct MaintenanceTask **tasks, int *taskCount);
void displayTasks(struct MaintenanceTask *tasks, int taskCount);

int main()
{
   struct MaintenanceTask *tasks = NULL;
   int taskCount = 0, option;
   do
   {
      printf("\nShip Maintenance Scheduler\n");
      printf("1. Add Maintenance Task\n");
      printf("2. Display All Tasks\n");
      printf("3. Exit\n");
      printf("Enter the option: ");
      scanf("%d", &option);
      switch (option)
      {
         case 1: addTask(&tasks, &taskCount); break;
         case 2: displayTasks(tasks, taskCount); break;
         case 3: for (int i = 0; i < taskCount; i++)
                    free((char *)tasks[i].taskID);
                 free(tasks);
                 printf("Exiting the scheduler\n");
                 break;
         default: printf("Invalid option\n");
```

```
        }
    } while (option != 3);
    return 0;
}

void addTask(struct MaintenanceTask **tasks, int *taskCount)
{
    char tempTaskID[50], tempDescription[200], tempSchedule[100];
    printf("Enter Task ID: ");
    scanf(" %[^\n]", tempTaskID);
    printf("Enter Task Description: ");
    scanf(" %[^\n]", tempDescription);
    printf("Enter Task Schedule (Date and Time): ");
    scanf(" %[^\n]", tempSchedule);
    struct MaintenanceTask newTask;
    newTask.taskID = (char *)malloc((strlen(tempTaskID) + 1) * sizeof(char));
    strcpy((char *)newTask.taskID, tempTaskID);
    strcpy(newTask.description, tempDescription);
    strcpy(newTask.schedule, tempSchedule);
    *tasks = (struct MaintenanceTask *)realloc(*tasks, (*taskCount + 1) *
                                               sizeof(struct MaintenanceTask));
    (*tasks)[*taskCount] = newTask;
    (*taskCount)++;
    printf("Task added successfully\n");
}

void displayTasks(struct MaintenanceTask *tasks, int taskCount)
```

```
{
    if (taskCount == 0)
    {
        printf("No tasks available\n");
        return;
    }
    printf("\nScheduled Maintenance Tasks\n");
    for (int i = 0; i < taskCount; i++)
    {
        printf("Task %d:\n", i + 1);
        printf("Task ID: %s\n", tasks[i].taskID);
        printf("Description: %s\n", tasks[i].description);
        printf("Schedule: %s\n", tasks[i].schedule);
    }
}
```

4. Cargo Loading Optimization

Description:
Design a system to optimize cargo loading using arrays for storing cargo weights and structures for vessel specifications. Unions represent variable cargo properties like dimensions or temperature requirements.

Specifications:

- Structure: Vessel specifications (capacity, dimensions).
- Union: Cargo properties (weight, dimensions).
- Array: Cargo data.
- const Pointers: Protect cargo data.
- Double Pointers: Dynamic cargo list allocation.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Vessel
{
    const char *vesselID;
    float length, width, height;
};

union CargoProperties
{
    int weight;
    struct
    {
        float length, width, height;
    } dimensions;
    struct
    {
        float temperature;
    } temperatureReqs;
};

struct Cargo
{
    const char *cargoID;
    union CargoProperties properties;
```

```
    int flag;
};

void addVessel(struct Vessel **fleet, int *fleetCount);
void addCargo(struct Cargo **cargoList, int *cargoCount);
void displayFleet(struct Vessel *fleet, int fleetCount);
void displayCargo(struct Cargo *cargoList, int cargoCount);

int main()
{
    struct Vessel *fleet = NULL;
    struct Cargo *cargoList = NULL;
    int fleetCount = 0, cargoCount = 0, option;
    do
    {
        printf("\nCargo Loading Optimization System\n");
        printf("1. Add Vessel\n");
        printf("2. Add Cargo\n");
        printf("3. Display Fleet\n");
        printf("4. Display Cargo List\n");
        printf("5. Exit\n");
        printf("Enter the option: ");
        scanf("%d", &option);
        switch (option)
        {
            case 1: addVessel(&fleet, &fleetCount); break;
            case 2: addCargo(&cargoList, &cargoCount); break;
```

```
            case 3: displayFleet(fleet, fleetCount); break;
            case 4: displayCargo(cargoList, cargoCount); break;
            case 5: for (int i = 0; i < fleetCount; i++)
                        free((char *)fleet[i].vesselID);
                    free(fleet);
                    for (int i = 0; i < cargoCount; i++)
                        free((char *)cargoList[i].cargoID);
                    free(cargoList);
                    printf("Exiting the system\n");
                    break;
            default: printf("Invalid option\n");
        }
    } while (option != 5);
    return 0;
}

void addVessel(struct Vessel **fleet, int *fleetCount)
{
    char tempVesselID[50], tempName[100];
    printf("Enter Vessel ID: ");
    scanf(" %[^\n]", tempVesselID);
    struct Vessel newVessel;
    newVessel.vesselID = (char *)malloc((strlen(tempVesselID) + 1) *
                                                        sizeof(char));
    strcpy((char *)newVessel.vesselID, tempVesselID);
    printf("Enter Vessel Dimensions (Length x Width x Height): ");
    scanf("%f %f %f", &newVessel.length, &newVessel.width,
```

```
                                                &newVessel.height);
    *fleet = (struct Vessel *)realloc(*fleet, (*fleetCount + 1) *
                                                        sizeof(struct Vessel));
    (*fleet)[*fleetCount] = newVessel;
    (*fleetCount)++;
    printf("Vessel added successfully\n");
}

void addCargo(struct Cargo **cargoList, int *cargoCount)
{
    char tempCargoID[50], tempType[50];
    int t;
    printf("Enter Cargo ID: ");
    scanf(" %[^\n]", tempCargoID);
    printf("Select Type (1 for perishable, 0 for non-perishable): ");
    scanf("%d", &t);
    struct Cargo newCargo;
    newCargo.cargoID = (char *)malloc((strlen(tempCargoID) + 1) *
                                                            sizeof(char));
    strcpy((char *)newCargo.cargoID, tempCargoID);
    newCargo.flag = t;
    if (t)
    {
        printf("Enter Temperature Requirement: ");
        scanf("%f", &newCargo.properties.temperatureReqs.temperature);
    }
    else
```

```
    {
        printf("Enter Cargo Weight: ");
        scanf("%d", &newCargo.properties.weight);
        printf("Enter Cargo Dimensions (Length x Width x Height): ");
        scanf("%f %f %f", &newCargo.properties.dimensions.length,
                &newCargo.properties.dimensions.width,
                &newCargo.properties.dimensions.height);
    }
    *cargoList = (struct Cargo *)realloc(*cargoList, (*cargoCount + 1) *
                                                    sizeof(struct Cargo));
    (*cargoList)[*cargoCount] = newCargo;
    (*cargoCount)++;
    printf("Cargo added successfully\n");
}

void displayFleet(struct Vessel *fleet, int fleetCount)
{
    if (fleetCount == 0)
    {
        printf("No vessels available\n");
        return;
    }
    printf("\nFleet Details:\n");
    for (int i = 0; i < fleetCount; i++)
    {
        printf("Vessel %d:\n", i + 1);
        printf("Vessel ID: %s\n", fleet[i].vesselID);
        printf("Dimensions: %.2f x %.2f x %.2f meters\n", fleet[i].length,
```

```
                                        fleet[i].width, fleet[i].height);
    }
}

void displayCargo(struct Cargo *cargoList, int cargoCount)
{
    if (cargoCount == 0){
        printf("No cargo available\n");
        return;
    }
    printf("\nCargo List:\n");
    for (int i = 0; i < cargoCount; i++)
    {
        printf("Cargo %d:\n", i + 1);
        printf("Cargo ID: %s\n", cargoList[i].cargoID);
        if (cargoList[i].flag)
            printf("Temperature Requirement: %.2f°C\n",
                    cargoList[i].properties.temperatureReqs.temperature);
        else
        {
            printf("Weight: %d kg\n", cargoList[i].properties.weight);
            printf("Dimensions: %.2f x %.2f x %.2f meters\n",
                                cargoList[i].properties.dimensions.length,
                                cargoList[i].properties.dimensions.width,
                                cargoList[i].properties.dimensions.height);
        }
    }
}
```

5. Real-Time Weather Alert System

Description:
Develop a weather alert system for ships using strings for alert messages, structures for weather data, and arrays for historical records.

Specifications:

- Structure: Weather data (temperature, wind speed).
- Array: Historical records.
- Strings: Alert messages.
- const Pointers: Protect alert details.
- Double Pointers: Dynamic weather record management.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_ALERT_LENGTH 100

struct WeatherData
{
   float temperature;
   float windSpeed;
};

void addWeatherRecord(struct WeatherData **weatherRecords,
                                                    int *recordCount);
void displayWeatherHistory(struct WeatherData *weatherRecords,
                                                    int recordCount);
const char *alert(struct WeatherData *weather);
```

```
int main()
{
    struct WeatherData *weatherRecords = NULL;
    int recordCount = 0, option;
    do
    {
        printf("\nReal-Time Weather Alert System\n");
        printf("1. Add Weather Data\n");
        printf("2. Display Weather History\n");
        printf("3. Generate Weather Alert\n");
        printf("4. Exit\n");
        printf("Enter the option: ");
        scanf("%d", &option);
        switch (option)
        {
            case 1: addWeatherRecord(&weatherRecords, &recordCount); break;
            case 2: displayWeatherHistory(weatherRecords, recordCount); break;
            case 3: if (recordCount > 0)
                    {
                        for (int i = 0; i < recordCount; i++)
                            printf("Weather Alert %d: %s\n", i + 1,
                                    alert(&weatherRecords[i]));
                    }
                    else
                        printf("No weather records available\n");
                    break;
            case 4: free(weatherRecords);
```

```
                printf("Exiting the system\n");
                break;
            default: printf("Invalid option\n");
        }
    } while (option != 4);
    return 0;
}

void addWeatherRecord(struct WeatherData **weatherRecords,
                                                    int *recordCount)
{
    struct WeatherData newWeather;
    printf("Enter Temperature: ");
    scanf("%f", &newWeather.temperature);
    printf("Enter Wind Speed: ");
    scanf("%f", &newWeather.windSpeed);
    *weatherRecords = (struct WeatherData *)realloc(*weatherRecords,
                            (*recordCount + 1) * sizeof(struct WeatherData));
    (*weatherRecords)[*recordCount] = newWeather;
    (*recordCount)++;
    printf("Weather data added successfully\n");
}

void displayWeatherHistory(struct WeatherData *weatherRecords,
                                                    int recordCount)
{
    if (recordCount == 0)
```

```
    {
        printf("No weather records available\n");
        return;
    }
    printf("\nWeather History:\n");
    for (int i = 0; i < recordCount; i++)
    {
        printf("Record %d: Temperature = %.2f°C, Wind Speed = %.2f km/h\n",
               i + 1, weatherRecords[i].temperature, weatherRecords[i].windSpeed);
    }
}

const char *alert(struct WeatherData *weather)
{
    if (weather->temperature > 35.0)
        return "Extreme Heat Alert!";
    else if (weather->windSpeed > 100.0)
        return "Severe Storm Alert!";
    else if (weather->temperature < 0.0)
        return "Cold Weather Alert!";
    else
        return "Weather is normal";
}
```

6. Nautical Chart Management

Description:
Implement a nautical chart management system using arrays for coordinates and structures for chart metadata. Use unions for depth or hazard data.

Specifications:

- Structure: Chart metadata (ID, scale, region).
- Union: Depth or hazard data.
- Array: Coordinate points.
- const Pointers: Immutable chart IDs.
- Double Pointers: Manage dynamic charts.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_COORDINATES 50

struct ChartMetadata
{
   const char *chartID;
   float scale;
   char region[100];
};

union DepthOrHazard
{
   float depth;
   char hazard[100];
```

```
};

struct Coordinate
{
    float latitude;
    float longitude;
    union DepthOrHazard data;
    int flag;  // 0 for depth, 1 for hazard
};

void addChart(struct ChartMetadata **charts, int *chartCount);
void displayCharts(struct ChartMetadata *charts, int chartCount);
void addCoordinates(struct Coordinate **coordinates, int *coordCount);
void displayCoordinates(struct Coordinate *coordinates, int coordCount);

int main()
{
    struct ChartMetadata *charts = NULL;
    int chartCount = 0, option;
    do
    {
        printf("\nNautical Chart Management System\n");
        printf("1. Add Chart\n");
        printf("2. Display Charts\n");
        printf("3. Add Coordinates to a Chart\n");
        printf("4. Exit\n");
        printf("Enter the option: ");
```

```
scanf("%d", &option);
switch (option)
{
   case 1: addChart(&charts, &chartCount); break;
   case 2: displayCharts(charts, chartCount); break;
   case 3: int chartIndex;
           printf("Enter the chart index to add coordinates to (0 to %d): ",
                                                    chartCount - 1);
           scanf("%d", &chartIndex);
           if (chartIndex >= 0 && chartIndex < chartCount)
           {
              struct Coordinate *coordinates = NULL;
              int coordCount = 0;
              addCoordinates(&coordinates, &coordCount);
              displayCoordinates(coordinates, coordCount);
              free(coordinates);
           }
           else
              printf("Invalid chart index\n");
           break;
   case 4: for (int i = 0; i < chartCount; i++)
              free((char *)charts[i].chartID);
           free(charts);
           printf("Exiting the management\n");
           break;
   default: printf("Invalid option\n");
}
```

```
    } while (option != 4);
    return 0;
}

void addChart(struct ChartMetadata **charts, int *chartCount)
{
    struct ChartMetadata newChart;
    char tempChartID[50], tempRegion[100];
    printf("Enter Chart ID: ");
    scanf(" %[^\n]", tempChartID);
    printf("Enter Chart Scale: ");
    scanf("%f", &newChart.scale);
    printf("Enter Region: ");
    scanf(" %[^\n]", tempRegion);
    newChart.chartID = (char *)malloc((strlen(tempChartID) + 1) * sizeof(char));
    strcpy((char *)newChart.chartID, tempChartID);
    strcpy(newChart.region, tempRegion);
    *charts = (struct ChartMetadata *)realloc(*charts, (*chartCount + 1) *
                                                    sizeof(struct ChartMetadata));
    (*charts)[*chartCount] = newChart;
    (*chartCount)++;
    printf("Chart added successfully\n");
}

void displayCharts(struct ChartMetadata *charts, int chartCount)
{
    if (chartCount == 0)
```

```
    {
        printf("No charts available\n");
        return;
    }
    printf("\nList of Charts\n");
    for (int i = 0; i < chartCount; i++)
    {
        printf("Chart %d:\n", i + 1);
        printf("Chart ID: %s\n", charts[i].chartID);
        printf("Scale: 1:%.0f\n", charts[i].scale);
        printf("Region: %s\n", charts[i].region);
    }
}

void addCoordinates(struct Coordinate **coordinates, int *coordCount)
{
    struct Coordinate newCoordinate;
    printf("Enter Latitude: ");
    scanf("%f", &newCoordinate.latitude);
    printf("Enter Longitude: ");
    scanf("%f", &newCoordinate.longitude);
    printf("Enter 1 for hazard, 0 for depth: ");
    scanf("%d", &newCoordinate.flag);
    if (newCoordinate.flag)
    {
        printf("Enter Hazard Description: ");
        scanf(" %[^\n]", newCoordinate.data.hazard);
```

```
    }
    else
    {
        printf("Enter Depth: ");
        scanf("%f", &newCoordinate.data.depth);
    }
    *coordinates = (struct Coordinate *)realloc(*coordinates, (*coordCount + 1) *
                                                    sizeof(struct Coordinate));
    (*coordinates)[*coordCount] = newCoordinate;
    (*coordCount)++;
    printf("Coordinate added successfully\n");
}

void displayCoordinates(struct Coordinate *coordinates, int coordCount)
{
    if (coordCount == 0)
    {
        printf("No coordinates available\n");
        return;
    }
    printf("\nList of Coordinates\n");
    for (int i = 0; i < coordCount; i++)
    {
        printf("Coordinate %d:\n", i + 1);
        printf("Latitude: %.2f, Longitude: %.2f\n", coordinates[i].latitude,
                                                    coordinates[i].longitude);
        if (coordinates[i].flag)
```

```
            printf("Hazard: %s\n", coordinates[i].data.hazard);
        else
            printf("Depth: %.2f meters\n", coordinates[i].data.depth);
    }
}
```

7. Crew Roster Management

Description:
Develop a system to manage ship crew rosters using strings for names, arrays for schedules, and structures for roles.

Specifications:

- Structure: Crew details (name, role, schedule).
- Array: Roster.
- Strings: Crew names.
- const Pointers: Protect role definitions.
- Double Pointers: Dynamic roster allocation.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_SCHEDULE_SIZE 7

struct CrewMember
{
    const char *role;
    char name[100];
```

```
    char schedule[MAX_SCHEDULE_SIZE];
};

void addCrewMember(struct CrewMember **roster, int *rosterCount);
void displayRoster(struct CrewMember *roster, int rosterCount);

int main()
{
    struct CrewMember *roster = NULL;
    int rosterCount = 0, option;
    do
    {
        printf("\nCrew Roster Management System\n");
        printf("1. Add Crew Member\n");
        printf("2. Display Roster\n");
        printf("3. Exit\n");
        printf("Enter the option: ");
        scanf("%d", &option);
        switch (option)
        {
            case 1: addCrewMember(&roster, &rosterCount); break;
            case 2: displayRoster(roster, rosterCount); break;
            case 3: free(roster);
                    printf("Exiting the management\n");
                    break;
            default: printf("Invalid option\n");
        }
```

```
    } while (option != 3);
    return 0;
}

void addCrewMember(struct CrewMember **roster, int *rosterCount)
{
    struct CrewMember newCrewMember;
    char tempName[100];
    const char *roles[] = {"Captain", "Engineer", "Deckhand", "Cook",
                                                        "Navigator"};
    int numRoles = sizeof(roles) / sizeof(roles[0]);
    int roleChoice;
    printf("Enter Crew Member Name: ");
    scanf(" %[^\n]", tempName);
    printf("Available Roles\n");
    printf("0. Captain\n1. Engineer\n2. Deckhand\n3. Cook\n4. Navigator\n");
    printf("Select Role: ");
    scanf("%d", &roleChoice);
    if (roleChoice < 0 || roleChoice >= 5)
    {
        printf("Invalid role\n");
        return;
    }
    strcpy(newCrewMember.name, tempName);
    newCrewMember.role = (char *)roles[roleChoice];
    printf("Enter schedule (1 for working, 0 for off): ");
    scanf("%s", newCrewMember.schedule);
```

```
    *roster = (struct CrewMember *)realloc(*roster, (*rosterCount + 1) *
                                                    sizeof(struct CrewMember));
    (*roster)[*rosterCount] = newCrewMember;
    (*rosterCount)++;
    printf("Crew member added successfully\n");
}

void displayRoster(struct CrewMember *roster, int rosterCount)
{
    if (rosterCount == 0)
    {
        printf("No crew members available\n");
        return;
    }
    printf("\nCrew Roster\n");
    for (int i = 0; i < rosterCount; i++)
    {
        printf("Crew Member %d:\n", i + 1);
        printf("Name: %s\n", roster[i].name);
        printf("Role: %s\n", roster[i].role);
        printf("Schedule: %s\n", roster[i].schedule);
    }
}
```

8. Underwater Sensor Monitoring

Description:
Create a system for underwater sensor monitoring using arrays for readings, structures for sensor details, and unions for variable sensor types.

Specifications:

- Structure: Sensor details (ID, location).
- Union: Sensor types (temperature, pressure).
- Array: Sensor readings.
- const Pointers: Protect sensor IDs.
- Double Pointers: Dynamic sensor lists.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_READINGS 3

struct Sensor
{
   const char *sensorID;
   char location[100];
   union SensorType
   {
      float temperature;
      float pressure;
   } data;
   int type;   // 1 for Temperature, 0 for Pressure
   float readings[MAX_READINGS];
```

```
};

void addSensor(struct Sensor **sensorList, int *sensorCount);
void displaySensors(struct Sensor *sensorList, int sensorCount);

int main()
{
   struct Sensor *sensorList = NULL;
   int sensorCount = 0, option;
   do
   {
      printf("\nUnderwater Sensor Monitoring System\n");
      printf("1. Add Sensor\n");
      printf("2. Display Sensors\n");
      printf("3. Exit\n");
      printf("Enter the option: ");
      scanf("%d", &option);
      switch (option)
      {
         case 1: addSensor(&sensorList, &sensorCount); break;
         case 2: displaySensors(sensorList, sensorCount); break;
         case 3: for (int i = 0; i < sensorCount; i++)
                    free((char *)sensorList[i].sensorID);
                 free(sensorList);
                 printf("Exiting the system\n");
                 break;
         default: printf("Invalid option\n");
```

```
        }
    } while (option != 3);
    return 0;
}

void addSensor(struct Sensor **sensorList, int *sensorCount)
{
    struct Sensor newSensor;
    char tempSensorID[50], tempLocation[100];
    int sensorType, i;
    printf("Enter Sensor ID: ");
    scanf(" %[^\n]", tempSensorID);
    printf("Enter Sensor Location: ");
    scanf(" %[^\n]", tempLocation);
    printf("Select Sensor Type (1 for Temperature, 0 for Pressure): ");
    scanf("%d", &sensorType);
    newSensor.sensorID = (char *)malloc((strlen(tempSensorID) + 1) *
                                                        sizeof(char));
    strcpy((char *)newSensor.sensorID, tempSensorID);
    strcpy(newSensor.location, tempLocation);
    newSensor.type = sensorType;
    if (sensorType)
    {
        printf("Enter Temperature Reading: ");
        scanf("%f", &newSensor.data.temperature);
    }
    else
```

```
    {
        printf("Enter Pressure Reading: ");
        scanf("%f", &newSensor.data.pressure);
    }
    printf("Enter %d readings for the sensor: \n", MAX_READINGS);
    for (i = 0; i < MAX_READINGS; i++)
    {
        printf("Reading %d: ", i + 1);
        scanf("%f", &newSensor.readings[i]);
    }
    *sensorList = (struct Sensor *)realloc(*sensorList, (*sensorCount + 1) *
                                                    sizeof(struct Sensor));
    (*sensorList)[*sensorCount] = newSensor;
    (*sensorCount)++;
    printf("Sensor added successfully\n");
}

void displaySensors(struct Sensor *sensorList, int sensorCount)
{
    if (sensorCount == 0)
    {
        printf("No sensors available\n");
        return;
    }
    printf("\nSensor Details\n");
    for (int i = 0; i < sensorCount; i++)
    {
```

```
        printf("Sensor %d:\n", i + 1);
        printf("Sensor ID: %s\n", sensorList[i].sensorID);
        printf("Location: %s\n", sensorList[i].location);
        if (sensorList[i].type)
        {
            printf("Sensor Type: Temperature\n");
            printf("Temperature: %.2f°C\n", sensorList[i].data.temperature);
        }
        else
        {
            printf("Sensor Type: Pressure\n");
            printf("Pressure: %.2f Pa\n", sensorList[i].data.pressure);
        }
        printf("Readings:\n");
        for (int j = 0; j < MAX_READINGS; j++)
            printf("    Reading %d: %.2f\n", j + 1, sensorList[i].readings[j]);
    }
}
```

9. Ship Log Management

Description:
Design a ship log system using strings for log entries, arrays for daily records, and structures for log metadata.

Specifications:

- Structure: Log metadata (date, author).
- Array: Daily log records.
- Strings: Log entries.

- const Pointers: Immutable metadata.
- Double Pointers: Manage dynamic log entries.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_LOG_ENTRIES 5

struct LogMetadata
{
   const char *date;
   const char *author;
};

struct LogEntry
{
   struct LogMetadata metadata;
   char entries[MAX_LOG_ENTRIES][100];
   int entryCount;
};

void addLogEntry(struct LogEntry **logs, int *logCount);
void displayLogs(struct LogEntry *logs, int logCount);

int main()
{
```

```
    struct LogEntry *logs = NULL;
    int logCount = 0, option;
    do
    {
        printf("\nShip Log Management System\n");
        printf("1. Add Log Entry\n");
        printf("2. Display Logs\n");
        printf("3. Exit\n");
        printf("Enter the option: ");
        scanf("%d", &option);
        switch (option)
        {
            case 1: addLogEntry(&logs, &logCount); break;
            case 2: displayLogs(logs, logCount); break;
            case 3: for (int i = 0; i < logCount; i++)
                    {
                        free((char *)logs[i].metadata.date);
                        free((char *)logs[i].metadata.author);
                    }
                    free(logs);
                    printf("Exiting the system\n");
                    break;
            default: printf("Invalid option\n");
        }
    } while (option != 3);
    return 0;
}
```

```
void addLogEntry(struct LogEntry **logs, int *logCount)
{
    struct LogEntry newLog;
    char tempDate[20], tempAuthor[100];
    int entryIndex;
    printf("Enter Log Date: ");
    scanf(" %[^\n]", tempDate);
    printf("Enter Author Name: ");
    scanf(" %[^\n]", tempAuthor);
    newLog.metadata.date = (char *)malloc((strlen(tempDate) + 1) * sizeof(char));
    strcpy((char *)newLog.metadata.date, tempDate);
    newLog.metadata.author = (char *)malloc((strlen(tempAuthor) + 1) *
                                                        sizeof(char));
    strcpy((char *)newLog.metadata.author, tempAuthor);
    newLog.entryCount = 0;
    printf("Enter up to %d log entries for this day:\n", MAX_LOG_ENTRIES);
    for (entryIndex = 0; entryIndex < MAX_LOG_ENTRIES; entryIndex++)
    {
        char tempEntry[256];
        printf("Enter Log Entry %d: ", entryIndex + 1);
        getchar();
        if (scanf("%[^\n]", tempEntry) == 0)
            break;
        getchar();
        strcpy(newLog.entries[entryIndex], tempEntry);
        newLog.entryCount++;
    }
```

```
    *logs = (struct LogEntry *)realloc(*logs, (*logCount + 1) *
                                                    sizeof(struct LogEntry));
    (*logs)[*logCount] = newLog;
    (*logCount)++;
    printf("Log entry added successfully\n");
}

void displayLogs(struct LogEntry *logs, int logCount)
{
    if (logCount == 0)
    {
        printf("No logs available\n");
        return;
    }
    printf("\nShip Log Details\n");
    for (int i = 0; i < logCount; i++)
    {
        printf("Log %d:\n", i + 1);
        printf("Date: %s\n", logs[i].metadata.date);
        printf("Author: %s\n", logs[i].metadata.author);
        printf("Entries:\n");
        for (int j = 0; j < logs[i].entryCount; j++)
            printf("  %d. %s\n", j + 1, logs[i].entries[j]);
    }
}
```

10. Navigation Waypoint Manager

Description:
Develop a waypoint management tool using arrays for storing waypoints, strings for waypoint names, and structures for navigation details.

Specifications:

- Structure: Navigation details (ID, waypoints).
- Array: Waypoint data.
- Strings: Names of waypoints.
- const Pointers: Protect waypoint IDs.
- Double Pointers: Dynamic waypoint storage.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct NavigationDetails
{
   const char *waypointID;
   char name[100];
};

void addWaypoint(struct NavigationDetails **waypoints, int *waypointCount);
void displayWaypoints(struct NavigationDetails *waypoints,
                                                    int waypointCount);

int main()
{
   struct NavigationDetails *waypoints = NULL;
```

```
    int waypointCount = 0, option;
    do
    {
        printf("\nNavigation Waypoint Manager\n");
        printf("1. Add Waypoint\n");
        printf("2. Display Waypoints\n");
        printf("3. Exit\n");
        printf("Enter the option: ");
        scanf("%d", &option);
        switch (option)
        {
            case 1: addWaypoint(&waypoints, &waypointCount); break;
            case 2: displayWaypoints(waypoints, waypointCount); break;
            case 3:  for (int i = 0; i < waypointCount; i++)
                        free((char *)waypoints[i].waypointID);
                    free(waypoints);
                    printf("Exiting\n");
                    break;
            default: printf("Invalid option\n");
        }
    } while (option != 3);
    return 0;
}

void addWaypoint(struct NavigationDetails **waypoints, int *waypointCount)
{
    struct NavigationDetails newWaypoint;
```

```
    char tempID[50], tempName[100];
    printf("Enter Waypoint ID: ");
    scanf(" %[^\n]", tempID);
    printf("Enter Waypoint Name: ");
    scanf(" %[^\n]", tempName);
    newWaypoint.waypointID = (char *)malloc((strlen(tempID) + 1) *
                                                        sizeof(char));
    strcpy((char *)newWaypoint.waypointID, tempID);
    strcpy(newWaypoint.name, tempName);
    *waypoints = (struct NavigationDetails *)realloc(*waypoints,
                        (*waypointCount + 1) * sizeof(struct NavigationDetails));
    (*waypoints)[*waypointCount] = newWaypoint;
    (*waypointCount)++;
    printf("Waypoint added successfully\n");
}

void displayWaypoints(struct NavigationDetails *waypoints, int waypointCount)
{
    if (waypointCount == 0)
    {
        printf("No waypoints available\n");
        return;
    }
    printf("\nWaypoint Details\n");
    for (int i = 0; i < waypointCount; i++)
    {
        printf("Waypoint %d:\n", i + 1);
```

```
        printf("Waypoint ID: %s\n", waypoints[i].waypointID);
        printf("Name: %s\n", waypoints[i].name);
    }
}
```

11. Marine Wildlife Tracking

Description:
Create a system for tracking marine wildlife using structures for animal data and arrays for observation records.

Specifications:

- Structure: Animal data (species, ID, location).
- Array: Observation records.
- Strings: Species names.
- const Pointers: Protect species IDs.
- Double Pointers: Manage dynamic tracking data.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct AnimalData
{
    const char *speciesID;
    char speciesName[100];
    char location[100];
};
```

```
void addObservation(struct AnimalData **observations, int *observationCount);
void displayObservations(struct AnimalData *observations,
                                                    int observationCount);

int main()
{
   struct AnimalData *observations = NULL;
   int observationCount = 0, option;
   do
   {
      printf("\nMarine Wildlife Tracking System\n");
      printf("1. Add Observation\n");
      printf("2. Display Observations\n");
      printf("3. Exit\n");
      printf("Enter the option: ");
      scanf("%d", &option);
      switch (option)
      {
         case 1: addObservation(&observations, &observationCount); break;
         case 2: displayObservations(observations, observationCount); break;
         case 3: for (int i = 0; i < observationCount; i++)
                     free((char *)observations[i].speciesID);
                 free(observations);
                 printf("Exiting the system.\n");
                 break;
         default: printf("Invalid option\n");
      }
```

```
    } while (option != 3);
    return 0;
}

void addObservation(struct AnimalData **observations, int *observationCount)
{
    struct AnimalData newObservation;
    char tempSpeciesID[50], tempSpeciesName[100], tempLocation[100];
    printf("Enter Species ID: ");
    scanf(" %[^\n]", tempSpeciesID);
    printf("Enter Species Name: ");
    scanf(" %[^\n]", tempSpeciesName);
    printf("Enter Location: ");
    scanf(" %[^\n]", tempLocation);
    newObservation.speciesID = (char *)malloc((strlen(tempSpeciesID) + 1) *
                                                        sizeof(char));
    strcpy((char *)newObservation.speciesID, tempSpeciesID);
    strcpy(newObservation.speciesName, tempSpeciesName);
    strcpy(newObservation.location, tempLocation);
    *observations = (struct AnimalData *)realloc(*observations,
                        (*observationCount + 1) * sizeof(struct AnimalData));
    (*observations)[*observationCount] = newObservation;
    (*observationCount)++;
    printf("Observation added successfully\n");
}

void displayObservations(struct AnimalData *observations,
```

```
                                                int observationCount)
{
    if (observationCount == 0)
    {
        printf("No observations available\n");
        return;
    }
    printf("\nWildlife Observation Details:\n");
    for (int i = 0; i < observationCount; i++)
    {
        printf("Observation %d:\n", i + 1);
        printf("Species ID: %s\n", observations[i].speciesID);
        printf("Species Name: %s\n", observations[i].speciesName);
        printf("Location: %s\n", observations[i].location);
    }
}
```